

数据结构讲义

qwaszx

2026 年 5 月 23 日

前言

有一句著名的话叫做“程序 = 算法 + 数据结构”。我们使用算法来描述解决问题的步骤，而数据结构提供了**实现步骤**的方法。在学习数据结构时，首先应当明确的事情即是它支持的**操作**：以什么样的数据为输入，允许什么样的修改，能够回答什么样的查询。反过来，面对一个问题时，我们也要能够根据需要的操作选择合适的数据结构。

一般来说，数据结构的功能越强大，其代价也就越大。这种代价可以是多方面的：时空复杂度、实现难度、常数等等。也许 OIer 首次接触到通用、强大的方法就是在学习数据结构之时。在惊叹于强力的通用方法的同时，不应忽视那些功能较弱但代价也更低的方法，更不能放弃对问题本身的分析。挖掘问题的性质以使用更精练的方法，忽略问题的性质而使用强力的通用方法，两者都是处理问题时的重要思想。

目录

第一章 基础数据结构	4
1.1 数组	4
1.2 <code>vector</code>	4
1.2.1 基本操作	4
1.2.2 常见问题	4
1.2.3 实现	5
1.3 链表	5
1.3.1 基本操作	5
1.4 栈和队列	5
1.5 双端队列	5
1.5.1 维护信息	6
1.6 二叉堆	6
1.6.1 基本操作	6
1.7 扩展类问题	7
1.8 单调队列和单调栈	8
第二章 静态区间数据结构	11
2.1 分治信息	11
2.2 静态区间数据结构	15
2.2.1 幂等律与 ST 表	15
2.2.2 分治与二区间合并	16
2.3 线段树	18
2.4 多叉分治	23
2.4.1 应用：线性 RMQ	23
2.5 * 理论最优的静态区间数据结构	24
第三章 带修区间数据结构	25
3.1 单点修改	25
3.2 全局修改	26
3.2.1 全局查询	26
3.2.2 区间查询	27
3.3 区间修改	30
3.4 特殊情况的优化	35
3.4.1 标记永久化	35
3.4.2 增量修改与前缀查询：树状数组	36
3.4.3 可差分的修改	37

第四章 线段树的进阶操作	39
4.1 zkw 线段树	39
4.2 动态开点	41
4.3 线段树上二分	43
4.4 线段树分裂与合并	44
4.5 单侧递归	44
4.6 李超线段树	44
4.7 Segment Tree Beats	44
4.8 KTT	44
第五章 扫描线与几何模型	45
5.1 平面数点问题	45
5.1.1 可减信息	45
5.1.2 不可减信息	46
5.1.3 转置	46
5.2 矩形并问题	46
5.3 常见平面建模	47
5.4 区间问题	47
5.4.1 数颜色类问题	47
5.4.2 mex 类问题	48
5.4.3 子区间类问题	49
5.5 维护时间维	50
第六章 分治	51
6.1 偏序数点	51
6.1.1 半在线	52

1 基础数据结构

本章提要

- `vector`、栈、队列、链表、堆支持的操作，及其实现
- 常见的可以使用这些数据结构解决的问题

提到复杂度时默认指最坏复杂度

1.1 数组

相信大家都会使用常数长度的数组。但是也有开变量长度的数组的方法：`int* a=new int[n]`。

通过这种方法，我们可以开每维长度不同的二维数组：

很多编译器也支持直接定义 `a[n]`，但这不是标准的。

```
1 int pool[1000000]; // 数组充当内存池
2 int* a[1000]; // 定义一个指针数组充当二维数组
3 int* cur=pool; // 内存池指针
4 for (int i=0; i<1000; i++) {
5     // a[i] 占用 [cur, cur+L[i]) 这一段空间
6     a[i]=cur;
7     cur+=L[i];
8 }
9 a[i][j]; // 访问 a[i][j]
```

相比 `vector` 套 `vector`，这种方法的内存是连续的，且常数更小。

1.2 vector

变长数组。基本可以完全替代数组。

1.2.1 基本操作

- 均摊 $O(1)$ 的末尾插入 `push_back`
- 均摊 $O(1)$ 的末尾删除 `pop_back`
- $O(1)$ 的随机访问¹`a[i]`
- $O(\text{size} - i)$ 加均摊 $O(1)$ 在第 i 个位置插入删除

参考：<https://cppreference.cn/w/cpp/container/vector>

1.2.2 常见问题

在使用时，需要注意一些陷阱：

¹指任意访问某个 `a[i]`

- 大量 `push_back` 会带来很大的常数。可以使用 `resize` 一次性分配需要的长度，这样和数组基本一样快
- 迭代器失效：修改导致长度 `size` 变化后，迭代器可能变得不可用
- `pop_back` 不会释放内存，所以占用空间正比于历史最大 `size` 而不是当前 `size`
- `clear` 不会释放内存。可以用 `vector<T>().swap(a)` 来释放

1.2.3 实现

我们给出一种 `vector` 的实现：如果 `size` 超过了当前容量，那么把当前容量加倍，重新申请空间，把所有东西都搬过去。

每次重新分配后，容量约等于两倍 `size`。触发下一次重新分配需要 $\Omega(\text{size})$ 次修改，重新分配的代价是 $O(\text{size})$ 。所以，平均每次修改的代价是 $O(1)$ 。

问题 1.1. 如果额外要求空间线性于 `size` 而不是历史最大 `size` 呢？

解答. `size` 小于当前容量的 $1/4$ 时，把容量减半并重新分配空间（为什么是 $1/4$ 而不是 $1/2$ ？）

1.3 链表

1.3.1 基本操作

链表支持

- $O(1)$ 在指定元素处插入删除单个元素/另一个链表（例如可以实现 $O(1)$ 拼接）
- $O(\text{位置})$ 随机访问
- $O(\text{size})$ 的空间

例 1.2. 洛谷 P10466 邻值查找 / 洛谷 P1168 中位数

插入往往需要平衡树之类的数据结构，但是如果倒着来就只需要先排序然后一直删除。如果排序可以线性，那么就能做到线性。

评注. 值域为 $n^{O(1)}$ 的排序可以通过多关键字计数排序，或者说基数排序来做到线性。

1.4 栈和队列

基本操作是广为人知的，不再赘述。显然，栈在功能上弱于 `vector`，因此可以用 `vector` 实现栈。而队列则不能直接用 `vector` 实现。如果队列大小已知不超过某个上界，那么可以用数组模拟循环队列。而如果想做到 $O(\text{size})$ 空间，就需要用链表或者双端队列实现

1.5 双端队列

双端队列在功能上强于 `vector` 和队列，但是实现更复杂，常数也更大。双端队列的支持的操作如下：

- 均摊 $O(1)$ 头尾插入删除
- $O(1)$ 随机访问

注意如果没有随机访问的要求，那么可以直接用链表完成。

STL 的 `deque` 即是标准的双端队列。然而，需要开很多双端队列时，要谨慎使用 STL 的 `deque`，因为一个空 `deque` 可能占用几百到几千字节的空间。

更好的实现是对顶栈。设想有两个栈，其底部粘在一起。于是其整体就成为一个两端开口的双端队列。唯一的问题出在某个栈空了还要继续删的情况。此时，我们暴力把整个序列倒出来重构即可。换句话说，我们重新把序列均分到两个栈里。

使用 `vector` 来实现栈就能支持随机访问了。如果需要很紧的空间，可以用之前提到的 $O(\text{size})$ 空间的 `vector`。

每次重构后，两个 `vector` 都有大概 $\text{size}/2$ 个元素。到下次重构之前，至少有 $\text{size}/2 + |\text{size}' - \text{size}/2| = \Omega(\text{size}')$ 次操作。重构花费 $O(\text{size}')$ 的代价，所以插入删除是均摊 $O(1)$ 的。

NOI 2022 时有许多人因为 MLE 失去了一整道题的分数。

1.5.1 维护信息

在两个 `vector` 中各自维护自己的前缀信息，这样可以 $O(1)$ 查询当前双端队列中跨过分断点的任意区间的分治信息。

分治信息的定义和例子将在下一章中给出。

例 1.3. 支持双端队列操作、每次询问以体积 V_i 做 01 背包的结果。需要 $O((n+q)V)$ 。

假设当前询问区间是 $[l, r]$ ，那么通过双端队列操作，我们可以 $O(1)$ 移动到 $[l \pm 1, r \pm 1]$ 。这样，我们就可以通过移动区间端点来在 $\sum_i |l_i - l_{i-1}| + |r_i - r_{i-1}|$ 的时间内回答多个区间询问。在很多问题中，这个总代价往往可以是线性的。

例 1.4. 滑动窗口。 给定序列 a 和整数 k ，查询所有 $[i, i+k-1]$ 的分治信息（如最大子段和）。

例 1.5. “不删除双指针”。 询问最短的 $\text{gcd} > 1$ 的区间。 $\text{gcd} > 1$ 可以换成 $\max \geq x$ 、按位或 $\geq x$ 之类的有单调性的条件。

1.6 二叉堆

1.6.1 基本操作

- $O(1)$ 查询最小值
- $O(\log n)$ 插入
- $O(\log n)$ 删除
- $O(\log n)$ Decrease-Key
- $O(n)$ 合并

评注. 理论上最好的堆——*Brodal Queue* 支持

- $O(1)$ 查询最小值

- $O(1)$ 插入
- $O(\log n)$ 删除
- $O(1)$ Decrease-Key
- $O(1)$ 合并

但是实现复杂、常数很大。

算法/数据结构总是在复杂度、实际表现、实现难度、常数、功能之间做取舍。

还有其他一些常见的堆：左偏树、斜堆、随机堆、配对堆、二项堆、斐波那契堆。

1.7 扩展类问题

从若干个初始状态开始，每个状态可以扩展出若干个后继状态，保证扩展出来的状态的状态的权值小于原状态的状态的权值。每次取权值最小的状态扩展，重复若干次。可惜没时间讲。

通过一定的设计，很多求前 k 大/小的问题可以变成这类问题。使用堆来维护所有目前的状态，每次取出最小的一个来扩展，注意去重。扩展形成一张图，复杂度为 $O(M \log M)$ 或 $O(k \log k + M)$ （取决于堆），其中 M 为前 k 小的点邻接的边数。

如果每个点的扩展方式都是相同的，并且只有常数种扩展方式，并且扩展出来的状态的状态的权值关于原状态的状态的权值是单调的，那么可以使用队列替换堆来做到 $O(M)$ 。

例 1.6. 序列合并.

解答. 方法一： (i, j) 扩展出 $(i, j+1)$ ，初始状态为所有 $(i, 1)$

方法二： (i, j) 扩展出 $(i+1, j)$ 和 $(i, j+1)$ ，但是要去重

方法三： <https://www.cnblogs.com/zkyJuruo/p/18428124>

例 1.7. 蚯蚓.

解答. 首先处理掉“除了新的以外都加 q ”，这可以变成全局加 q 然后新的减掉 q 。

这样就是 x 扩展出 $\lfloor p(x+ tq) \rfloor - (t+1)q$ 和 $(x+ tq) - \lfloor p(x+ tq) \rfloor - (t+1)q$

符合我们用队列替换堆时的要求

例 1.8. 常数种非负边权的最短路. BFS 扩展时，从 x 扩展出 $O(1)$ 种固定的新状态可以使用队列替换堆。

例 1.9. 前 k 小子集和.

解答. 考虑一个 DFS 过程，枚举下一个选择的数在哪

然而这样会扩展出 $\Theta(n)$ 个状态

多叉树转二叉树：左儿子右兄弟（注意兄弟的顺序）

例 1.10. 限制大小的前 k 小子集和. 要求集合大小在 $[L, R]$ 内。

解答. 考虑怎么枚举 $\text{size} = m$ 的集合。假设选了 $a_{i_1}, \dots, a_{i_m}$, 转为枚举 $d_j = i_j - i_{j-1}$, 只需要 $\sum_j d_j \leq n - 1$ 。

初始所有 $d_j = 1$, 每次枚举下一个需要增加的 d_j 即可。

同样用儿子兄弟表示法转为二叉树

评注. 这类问题应该还有很大研究空间, 现在知道的有 [这篇](#) 和 [这篇](#)。有兴趣的同学可以自行研究或者和我一起讨论一下

例 1.11. 合并果子. 由于合并的结果总是越来越大的, 可以使用队列替换堆。

评注. 这就是 *Huffman* 编码。我们有结论

- *Huffman* 编码可以最小化 $\sum_i \text{dep}_i \cdot w_i$
- w_i 的深度大约为 $\log \frac{W}{w_i}$, 其中 $W = \sum w_i$

在不均匀分治 (如树剖、点分治、分治 *FFT* 等) 中, 可以使用 *Huffman* 树来降低复杂度或常数, 以后会提到。

1.8 单调队列和单调栈

令 $b_i = \max\{a_i, a_{i+1}, \dots, a_r\}$, 即 b 是只考虑 $a_1, \dots, a_r$ 时的 a 的后缀 $\max$ 。我们可以在 r 从 1 扫到 n 的过程中均摊 $O(1)$ 地隐式维护所有 b_i 。具体来说, 维护一个栈, 栈中元素是 a 的一个子序列。假设自底向上的第 k 个元素是 a_{i_k} , 那么对任何 $i_{k-1} < j \leq i_k$, 都有 $b_j = a_{i_k}$, 也就是说栈中每个元素都表示一段以它为结尾的区间的 b_i 。由于 b 是不升的, 栈中元素也是不升的。当 r 移动到 $r+1$ 时, 考虑 a_{r+1} 的影响。假设当前栈中有 k 个元素。如果 $a_{r+1} < a_{i_k}$, 那么 a_{r+1} 不会对 $b_{[1,r]}$ 造成任何影响, 只需要把 a_{r+1} 加入栈中表示 b_{r+1} 即可。如果 $a_{r+1} \geq a_{i_k}$, 那么我们可以想象 $b_{[1,r]}$ 发生的变化: 一个后缀会全都变成 a_{r+1} 。在栈中, 这表现为弹出若干栈顶, 直到栈顶的 $a_{i_k} > a_{r+1}$, 最后再把 a_{r+1} 加入栈中。暴力做这个过程的均摊复杂度就是正确的, 因为每个 a_i 最多入栈出栈各一次。

练习 1.12. 把 $\max$ 换成 $\min$ 。

例 1.13. 询问端点单调 RMQ. 多次查询 $\max\{a_{l_i}, \dots, a_{r_i}\}$, 其中 l_i, r_i 都单调不降。可以强制在线 (指 a 可能依赖先前的询问结果)
虽然双端队列也可以做, 但是这种问题单调队列更方便。

解答. 单调栈中维护了所有的后缀 $\max$, 答案就是下标 $\geq l_i$ 的第一个元素。由于 l_i 不降, 我们直接弹出队头就可以均摊 $O(1)$ 地维护这个事情。

练习 1.14. 滑动窗口. 对每个 $1 \leq l \leq n - k + 1$ 查询 $\max\{a_i \mid i \in [l, l + k - 1]\}$ 。

练习 1.15. 在例 1.13 的基础上减弱问题的假设。

1. 去掉 l_i 单调的假设, 允许询问复杂度变大一些
2. 再通过离线去除 r_i 单调的假设

3. * 再通过可持久化数组支持在线
4. 在 $n \leq 64$ 时, 通过位运算来存储每个 r 的栈中元素集合, 从而实现 $O(n) - O(1)$ 的在线 RMQ。我们将在下一章中配合其他技巧做到任意 n 的线性 RMQ。

例 1.16. 求前驱后继. 对每个元素, 查询它左/右边第一个比它大/小的数的位置

解答. 右边就是把序列倒过来的左边, 所以只考虑左边。这就是单调栈中倒数第二个元素。

例 1.17. 稍复杂一些的信息维护. 对每个 r , 求 $\sum_{l=1}^r (r-l+1) \cdot \max\{a_l, \dots, a_r\}$ 。强制在线。(提示: 要记得单调栈中每个元素都代表了一段区间的 $\max\{a_l, \dots, a_r\}$ 。)

解答. 假设栈中元素是 $a_{i_1}, a_{i_2}, \dots, a_{i_k}$, 其中 $i_1 < i_2 < \dots < i_k$, 那么答案是

$$\begin{aligned} \sum_{j=1}^k a_{i_j} \sum_{i_{j-1} < l \leq i_j} (r-l+1) &= \sum_{j=1}^k a_{i_j} \cdot (i_j - i_{j-1}) \cdot (r - i_j + 1) \\ &= (r+1) \sum_{j=1}^k a_{i_j} - \sum_{j=1}^k a_{i_j} (S_{i_j} - S_{i_{j-1}}), \end{aligned}$$

其中 $S_i = \sum_{k=1}^i k$ 。直接维护这个式子即可。

离线的另解. 如果允许离线, 那么可以考虑换种方式计算答案: 对每个 a_i , 看看它是哪些区间 $[l, r]$ 的 $\max$ 。这可以由先前提到的前驱后继来做。

例 1.18. 最值版本. 求 $\max_{l,r} (r-l+1) \cdot \max\{a_l, \dots, a_r\}$ 。

解答. 使用离线算贡献的方法是最容易的

在线对每个 r 算的话需要维护凸包。

例 1.19. 悬线法. 有些题给了个二维黑白网格, 要求全黑的矩形/正方形的最大面积/面积和之类的东西

对每个格子计算每个方向全黑能延伸到哪, 然后就变成类似例三的问题

练习 1.20. 困难的例子 给定两个序列 a, b , 求有多少个区间 $[l, r]$ 满足 $\max a[l, r] = \max b[l, r]$ 。

解答. a, b 交替形成一个新序列, 在这个新序列上扫的时候用单调栈维护所有后缀 $\max$ 。在维护这个交替序列的后缀 $\max$ 的同时, 我们可以顺便维护出每个后缀 $\max$ 是否同时也是只考虑 a/b 的后缀 $\max$, 以及只考虑 a/b 时何时达到此后缀 $\max$ 。之后容易计算单调栈中每个项的贡献: 假设只考虑 a/b 时在原序列的 l_a/l_b 位置达到此后缀 $\max$, 则这一项的贡献即为 $\min(l_a, l_b) - \text{pos}$, 其中 pos 为此项在原序列中的位置, 另外在此项是由 b 产生的时候 pos 需要再加一。

也可以扩展到多个序列。

本章总结

- 熟知讲过的几种线性数据结构的功能，能够把问题与数据结构进行对应
- 扩展类问题的思路

2 静态区间数据结构

在本节中，我们将尝试解决一类这样的问题：给定一个长度为 n 的序列 a ，以及一个输入为区间 $[l, r]$ 和 a 的函数 f ，我们需要进行一定的预处理，使得对于任意的 l, r ，我们都可以快速地计算出 $f_{[l, r]}(a)$ 的值。这类问题通常称为静态区间信息查询。

2.1 分治信息

我们首先需要确定 f 的一个合适陪域 S 。如果 S 本身就是一个巨大的集合，那么我们自然无法期望得到一个快速的算法。

例 2.1. 假设 a 的所有元素都在集合 A 中。

- $f_{[l, r]}(a) = (a_l, \dots, a_r)$ 是一个平凡的无法快速查询的函数，因为它本身就需要写出所有的值。此时， $S = \{\epsilon\} + A + A \times A + A \times A \times A + \dots$ 是一个巨大的集合。
 - 假设 a 中的元素都在 $[V]$ 中（不超过 V 的正整数集），那么 $f_{[l, r]}(a) = \sum_{i=l}^r a_i$ 的陪域可以是 $[nV]$ ；而 $f_{[l, r]}(a) = \gcd(a_l, \dots, a_r)$ 和 $f_{[l, r]}(a) = \max(a_l, \dots, a_r)$ 的陪域都可以是 $[V]$ 。
 - 假设 a 中的元素都在 $\mathbb{F}_p$ 中（也就是 $\text{mod } p$ 下做运算），那么 $f_{[l, r]}(a) = \sum_{i=l}^r a_i$ 和 $f_{[l, r]}(a) = \prod_{i=l}^r a_i$ 的陪域都可以是 $\mathbb{F}_p$ 。
 - $f_{[l, r]}(a) = a_l \odot a_{l+1} \odot \dots \odot a_r$ 抽象了以上情况，其中 $\odot$ 是一个定义在 A 上的满足结合律的二元运算。此时， $S = A$ 。
- $f_{[l, r]}(a) = \bigoplus_{[i, j] \subseteq [l, r]} (a_i \odot \dots \odot a_j)$ 是另一个常见的情况，其中 $\odot$ 同上一条， $\oplus$ 是另一个定义在 A 上的满足交换律和结合律的二元运算（请注意 $\odot$ 和 $\oplus$ 在这里并不暗示分配律）。也就是说，枚举所有的子区间，子区间内做运算 $\odot$ ，然后再把所有子区间的结果用 $\oplus$ 合并。
- $f_{[l, r]}(a) =$ 区间众数的出现次数 是一个虽然 S 很小但也没有 polylog 做法的问题。

这是半群

但是在下面暗示了。

评注. 从信息论的角度看，至少需要 $\log |S|/w$ 个变量才能表示一个 S 中的元素。从 OI 的角度看， S 中每个元素都是表示成一个结构体的，那么存储 S 中的元素就需要花费结构体中的变量个数那么多的代价。

一般的 f 并不容易快速回答。但是， f 经常有一些好的性质。如果学过一些基本的线段树知识，那么可以发现我们在对询问区间进行切分，使得每个切出来的区间都被预处理过了，然后再用这些预处理的区间的 f 拼出询问区间的 f 。我们将此抽象为可分解性，这是最重要和常见的性质：

这是 GPT 起的名字

定义 2.2. 可分解性. 如果存在 S 上的二元运算 $\odot$ ，使得对任何 $l \leq m < r$ 都有 $f_{[l, r]}(a) = f_{[l, m]}(a) \odot f_{[m+1, r]}(a)$ ，则称 f 可分解。

满足可分解性的 f 一般称为分治信息。对于这样的 f ，我们可以用子区间的信息来拼出大区间的信息。

然而我也不知道
这通常被称为什么名字，也许这太平凡了

练习 2.3. 半群信息. 假设 f 可分解。

1. 假设 f 的值域为 $T \subseteq S$, 证明 $\odot$ 在 T 上满足结合律。
2. 证明 $f_{[l,r]}(a) = f_l(a) \odot f_{l+1}(a) \odot \cdots \odot f_r(a)$ 。
3. 说明为什么我们可以不失一般性地只考虑形如 $g_{[l,r]}(b) = b_l \odot b_{l+1} \odot \cdots \odot b_r$ 的问题。这也被称为半群信息。

练习 2.4. 么元. 假设存在一个么元 $0 \in S$ (注意这里的 0 是抽象的 0) 使得 $x \odot 0 = 0 \odot x = x$ 。此时, 我们可以避免一些边界上的麻烦。例如, 我们可以设 $f_{\varnothing}(a) = 0$, 从而在 $m = r$ 时也有 $f_{[l,r]}(a) = f_{[l,m]}(a) \odot f_{[m+1,r]}(a)$ 。
如果不存在么元, 说明我们总是可以人为补上一个。

下面是一些典型的分治信息的例子。

例 2.5.

- $f_{[l,r]}(a) = \{a_l\} \times \{a_{l+1}\} \times \cdots \times \{a_r\}$ 也是满足结合律的 (在同构意义下)! 然而, 我们显然不想用它来计算。
- $f_{[l,r]}(a) = a_l \odot \cdots \odot a_r$ 显然是可分解的。
- $f_{[l,r]}(a) = \bigoplus_{[i,j] \subseteq [l,r]} (a_i \odot \cdots \odot a_j)$ 并不直接可分解。但是, 如果 $\odot$ 和 $\oplus$ 满足分配律 $x \odot (y \oplus z) = (x \odot y) \oplus (x \odot z)$, 那么我们可以构造出一个包含 f 的可分解的函数。令

这是环

$$g_{[l,r]}(a) = \begin{pmatrix} f_{[l,r]}(a) \\ fl_{[l,r]}(a) \\ fr_{[l,r]}(a) \\ flr_{[l,r]}(a) \end{pmatrix} = \begin{pmatrix} \bigoplus_{[i,j] \subseteq [l,r]} (a_i \odot \cdots \odot a_j) \\ \bigoplus_{[l,j] \subseteq [l,r]} (a_l \odot \cdots \odot a_j) \\ \bigoplus_{[i,r] \subseteq [l,r]} (a_i \odot \cdots \odot a_r) \\ a_l \odot \cdots \odot a_r \end{pmatrix},$$

找出使得 $g_{[l,r]}(a) = g_{[l,m]} \otimes g_{[m+1,r]}$ 的运算 $\otimes$ 。(提示: $[l,r]$ 的子区间由三部分组成: $[l,m]$ 的子区间、 $[m+1,r]$ 的子区间、跨越 $[m,m+1]$ 的子区间, 最后一部分相当于由一个 $[l,m]$ 的后缀拼上一个 $[m+1,r]$ 的前缀。可以将 $(\oplus, \odot)$ 想象成 $(+, \times)$ 。)

评注. 有许多运算 $(\oplus, \odot)$ 都构成环, 也就是 $\oplus$ 交换结合、 $\odot$ 结合、 $\odot$ 对 $\oplus$ 有分配律。常见的例子有 $(+, \times)$, $(\max, +)$, $(\max, \min)$ 。这里 $\max$ 和 $\min$ 是等效的。还有很多运算相当于多维的 $\max$ 或 $\min$, 如位运算 $\&$ 和 $|$, lcm 和 gcd , 集合运算 $\cup$ 和 $\cap$ 。

另一方面, 环导出的矩阵乘法也是一个环。所以, 我们已经可以处理相当多的运算。

评注. 我们从这个例子中可以看到, 虽然原本的 f 无法直接处理, 但放到更大的结构 g 中反而就能够处理了。往小了说, 这是设计信息时的常见技巧: 补充缺少的元素, 直到结构变得封闭; 往大了说, 这是一种深刻的哲学: 发展新的概念和理论, 让困难的问题变得平凡。

「比起用锤头敲开核桃, 不如用海水把它泡软。」

一大类静态区间信息查询问题都可以在找到合适的分治信息后直接使用分治数据结构解决。我们在此非正式地表述寻找分治信息的一般思想。

原则 2.6. 分治信息的设计原则. 我们需要寻找一个可以回答询问 f 的分治信息。

- 如果 f 可以由一些更简单的函数组合出来 (如区间方差), 那么可以先找到这些更简

单的函数的分治信息，再将其组合。

- 观察 $f_{[l,r]}(a)$ 能否由 $f_{[l,m]}(a)$ 和 $f_{[m+1,r]}(a)$ 组合出来。如果不能，看看还需要哪些东西才能组合出来，把这些东西也加入信息中。
- 重复上一步若干次。如果顺利，那么可以得到一个分治信息。否则，可能说明不存在简单的分治信息。

让我们来做一些具体的练习，看看这个思想是如何使用的。

练习 2.7. 对以下询问，找到合适的能够回答询问的分治信息：

- 区间和 $a_l + \dots + a_r$
- 区间乘积 $a_l \dots a_r$ ，在 $\mathbb{F}_p$ 下做运算（也就是有一个全局固定的素数模数 p ）
- 区间最值 $\max(a_l, \dots, a_r)$
- 区间最大子段和 $\max_{[i,j] \subseteq [l,r]} (a_i + \dots + a_j)$
- 区间异或和 $a_l \oplus \dots \oplus a_r$
- 区间的所有子区间异或和的和 $\sum_{[i,j] \subseteq [l,r]} (a_i \oplus \dots \oplus a_j)$
- 区间方差 $\frac{1}{r-l+1} \sum_{i=l}^r (a_i - \bar{a}_{[l,r]})^2$ ，其中 $\bar{a}_{[l,r]} = \frac{1}{r-l+1} \sum_{i=l}^r a_i$
- 区间 k 次方和 $a_l^k + \dots + a_r^k$
- 区间位置乘值的和 $\sum_{i=l}^r i a_i$
- 区间两两乘积的和 $\sum_{l \leq i < j \leq r} a_i a_j$
- 区间内选 k 个数的乘积的和 $\sum_{l \leq i_1 < \dots < i_k \leq r} a_{i_1} \dots a_{i_k}$
- 区间相邻乘积的和 $\sum_{l \leq i < r} a_i a_{i+1}$
- 区间字符串哈希 $\sum_{l \leq i \leq r} a_i B^{r-i}$
- 区间 gcd: $\gcd(a_l, \dots, a_r)$
- 区间最长连续 0 长度 $\max_{[i,j] \subseteq [l,r]} (j - i + 1) [\forall k \in [i, j], a_k = 0]$
- 区间做括号匹配剩下的不匹配括号
- 区间内有多少个子区间同时包含 $1, 2, \dots, k$ （可以设计 $O(k)$ 大小的信息和 $O(k)$ 完成的运算）
- 区间众数，但是值域 k 很小

这里的 $\oplus$ 是异或而不是抽象运算。

在实现代码时，一种方便的写法是将 S 的表示封装进结构体，然后重载 S 上的 $+$ 运算符。然后，在写数据结构时，我们用 $+$ 即可完成「信息合并」。通过这种方式，我们解耦了数据结构与其维护的信息。

代码 2.1: 信息的封装

```

1 struct Info {
2     // 表示 S 中的一个元素需要的若干变量。我们以最大子段和为例。
3     long long s, sl, sr, slr;
4     // 默认构造函数：对应幺元 0。如果没有直接的表示，那么可以再加一个
        bool 变量表示。
5     Info() : s(-INF), sl(-INF), sr(-INF), slr(0) {}
6     // 构造函数，将序列的元素 x 转成 S 中的元素
7     Info(int x) : s(x), sl(x), sr(x), slr(x) {}

```

```

8
9 // 重载 + 运算符：实现运算 \odot
10 Info operator + (const Info &a, const Info &b)
11 {
12     Info res;
13     res.s = max(max(a.s, b.s), a.sl + b.sr);
14     res.sl = max(a.sl, a.slr + b.sl);
15     res.sr = max(b.sr, b.slr + a.sr);
16     res.slr = a.slr + b.slr;
17     return res;
18 }
19 };
20
21 // 在数据结构中使用示例：
22 // a[u] = Info(x);
23 // a[u] = a[u << 1] + a[u << 1 | 1];

```

有时, $\odot$ 是可减的, 也就是已知 $a \odot b$ 和 a 时可以解出 b 。此时, 我们可以通过求出 $f_{[1,r]}$ 和 $f_{[1,l-1]}$ 来解出 $f_{[l,r]}$ 。这样的问题可以通过前缀和来做到线性。

评注. 这个概念对应消去律。这和逆元、交换律都是不同的概念。

练习 2.8. 练习 2.7 中, 哪些是可逆的? 据此得到它们的线性做法。

最后一个有用的性质是**幂等律**, 它是说 $a \odot a = a$ 。典型的满足幂等律的运算是最值 $\max, \min$ 以及它们的多维版本。幂等律没有前面两个性质那么有用。在同时具备结合律的情况下, 幂等律允许我们把待查询的区间拆成可重叠的区间, 据此减少需要预处理的区间数量。具体来说, 我们有

$$\begin{aligned}
 \bigodot_{k=l}^r a_k &= \left(\bigodot_{k=l}^{i-1} a_k \right) \odot \left(\bigodot_{k=i}^j a_k \right) \odot \left(\bigodot_{k=j+1}^r a_k \right) \\
 &= \left(\bigodot_{k=l}^{i-1} a_k \right) \odot \left(\bigodot_{k=i}^j a_k \right) \odot \left(\bigodot_{k=i}^j a_k \right) \odot \left(\bigodot_{k=j+1}^r a_k \right) \\
 &= \left(\left(\bigodot_{k=l}^{i-1} a_k \right) \odot \left(\bigodot_{k=i}^j a_k \right) \right) \odot \left(\left(\bigodot_{k=i}^j a_k \right) \odot \left(\bigodot_{k=j+1}^r a_k \right) \right) \\
 &= \left(\bigodot_{k=l}^j a_k \right) \odot \left(\bigodot_{k=i}^r a_k \right)
 \end{aligned}$$

也就是说, 我们不光可以用 $f_{[l,i]}(a)$ 和 $f_{[i+1,r]}(a)$ 拼出 $f_{[l,r]}(a)$, 还可以用 $f_{[l,j]}(a)$ 和 $f_{[i,r]}(a)$ ($j \geq i$) 拼出 $f_{[l,r]}(a)$ 。

2.2 静态区间数据结构

在本节中，我们假设询问都是 $f[l, r] = a_l \odot \cdots \odot a_r$ 的形式，其中 $\odot$ 是有结合律的可以 $O(1)$ 计算的二元运算。我们在[练习 2.3](#)中解释了为什么可以这么做。

2.2.1 幂等律与 ST 表

我们先来展示如何对于额外满足幂等律的 $\odot$ 设计简单的数据结构来回答区间询问，也就是 ST 表 (Sparse Table)

数据结构 2.9. ST 表.

预处理. 对于所有满足 $1 \leq i + 2^k - 1 \leq n$ 的 i, k ，预处理 $st_k[i] = f[i, i + 2^k]$ 。这个递推是容易的： $st_0[i] = f[i] = a_i$ ， $st_k[i] = st_{k-1}[i] \odot st_{k-1}[i + 2^{k-1}]$ 。

询问. 询问 $f[l, r]$ 时，找到最大的 k 使得 $2^k \leq r - l + 1$ 。此时， $[l, l + 2^k)$ 和 $(r - 2^k, r]$ 可以覆盖整个区间 $[l, r]$ 。所以 $f[l, r] = st_k[l] \odot st_k[r - 2^k + 1]$ 。

现在来看一下复杂度。总共有 $O(n \log n)$ 个预处理的项，所以预处理时间为 $O(n \log n)$ 。每次查询只需要合并两个预处理的项，故查询复杂度 $O(1)$ 。对于寻找最大的 $k = \lfloor \log_2(r - l + 1) \rfloor$ ，可以预处理或者使用内置的 `__builtin_ctz`。

代码 2.2: ST 表

```

1  Info st[LOGN][N];
2  // O(nlogn) 预处理
3  void build() {
4      lg[1] = 0;
5      for (int i = 2; i <= n; i++) lg[i] = lg[i >> 1] + 1;
6
7      for (int i = 1; i <= n; i++) st[0][i] = Info(a[i]);
8      for (int k = 1; (1 << k) <= n; k++)
9          for (int i = 1; i + (1 << k) - 1 <= n; i++)
10             st[k][i] = st[k - 1][i] + st[k - 1][i + (1 << (k - 1))];
11 }
12 // O(1) 询问
13 Info query(int l, int r) {
14     int k = lg[r - l + 1]; // 也可以用 k=__builtin_ctz(r - l + 1);
15     return st[k][l] + st[k][r - (1 << k) + 1];
16 }

```

评注. 幂等律对于更低复杂度的数据结构设计并无太多帮助。然而，通过幂等律放松限制是一个重要的思想，这在最优化类型的动态规划中有时可以降低复杂度。

我们没有解释为什么可以假设能 $O(1)$ 计算。实际上，如果需要 $O(T)$ 计算，那么最后复杂度往往要再乘一个 T 。

翻译之后多出了一个表 (Table)。

这个函数是 GCC 私货。以前是 CCF 系列赛事原则上禁用的。

练习 2.10. 通过将 $[l, r]$ 拆成 $O(\log n)$ 个 $[p_i, p_i + 2^i)$ 的不交并, 说明 ST 表也可以在 $O(\log n)$ 的时间内查询不满足幂等律的信息。

练习 2.11. 说明 ST 表支持 $O(\log n)$ 的末尾插入。再拼合双端队列, 我们就能支持 $O(\log n)$ 的头尾插入。

2.2.2 分治与二区间合并

现在, 我们来考虑只有结合律的情况。一种想法是**分治**: 将目前待处理的区间 $[l, r]$ 分为两半 $[l, m]$ 和 $[m + 1, r]$, 那么询问有三种情况: 要么完全在左边, 要么完全在右边, 要么跨过中点。完全在某一边的情况可以变成子问题递归处理, 而对于跨过中点的询问, 我们可以将其变成一个 $[l, m]$ 上的后缀询问加上一个 $[m + 1, r]$ 上的前缀询问, 再用一次 $\odot$ 拼合。于是我们可以考虑预处理每个分治区间的这样的前后缀询问的结果, 这样就得到了以下的数据结构:

数据结构 2.12. 二区间合并.

预处理. 从 $[1, n]$ 开始分治。假设当前的分治区间为 $[l, r]$, 设其中点为 $m = \lfloor (l + r)/2 \rfloor$ 。如果 $l = r$, 那么结束; 否则, 预处理所有 $[l, m]$ 的后缀询问的结果以及所有 $[m + 1, r]$ 的前缀询问的结果, 然后递归处理 $[l, m]$ 和 $[m + 1, r]$ 。

查询. 对于查询 $[lq, rq]$, 从 $[1, n]$ 开始分治。假设当前的分治区间是 $[l, r]$, 中点为 $m = \lfloor (l + r)/2 \rfloor$ 。如果 $l = r$, 那么直接返回 a_l ; 如果 $[lq, rq]$ 完全在 $[l, m]$ 或者 $[m + 1, r]$, 则递归下去处理; 否则, 用预处理的前后缀计算 $f[lq, m] \odot f[m + 1, rq]$ 并返回。

现在来看一下复杂度。设长为 n 的分治区间需要花费 $T(n)$ 的时空预处理。那么左右两个子问题各自花费 $T(n/2)$ 的时空处理, 预处理前后缀询问花费 $O(n)$ 时空, 所以总的时空复杂度是 $T(n) = 2T(n/2) + O(n) = O(n \log n)$ 。对于询问, 代价是 $Q(n) = Q(n/2) + O(1) = O(\log n)$ 。

评注. 这个数据结构在询问时只使用了一次 $\odot$ 。在这个限制下, $\Omega(n \log n)$ 的预处理复杂度是必要的。

另外, 我们注意到询问的瓶颈事实上在于找到那个可以用前后缀回答的分治区间。我们将在练习中看到这个过程可以优化到 $O(1)$ 。

在 OI 圈中, 这个结构经常被叫做“猫树”。这是一个乱起名字的例子, 参见[此博客](#)。

代码 2.3: 二区间合并

```
1 Info a[N];
2 // 每一层分治对应一层数组
3 Info pre[LOGN][N], suf[LOGN][N];
4 void build(int l, int r, int dep) {
5     if (l == r) return;
6     int m = (l + r) >> 1;
7     suf[dep][m] = a[m];
8     for (int i = m - 1; i >= l; i--)
```

```

9      suf[dep][i] = a[i] + suf[dep][i + 1];    //注意：加号（实际代表 \
      odot）未必有交换律，所以两边不能反过来
10     pre[dep][m + 1] = a[m + 1];
11     for (int i = m + 2; i <= r; i++)
12         pre[dep][i] = pre[dep][i - 1] + a[i];
13     build(1, m, dep + 1);
14     build(m + 1, r, dep + 1);
15 }
16 // 查询 [lq, rq]
17 Info query(int l, int r, int lq, int rq, int dep) {
18     if (lq <= l && r <= rq) {
19         if (l == r) return a[l];
20     }
21     int m = (l + r) >> 1;
22     if (rq <= m) return query(1, m, lq, rq, dep + 1);
23     else if (lq > m) return query(m + 1, r, lq, rq, dep + 1);
24     else return suf[dep][lq] + pre[dep][rq];
25 }
26 Info query(int lq, int rq) {
27     return query(1, n, lq, rq, 0);
28 }

```

树与区间划分

上面这个代码并没有那么显然，它隐式地使用了一个事实：“一层”内的区间是不交的。为了更直观的理解这个事实，我们来考察区间划分与树的联系。对每个分治区间，我们建立一个结点在分治中，一个分治区间会被划分为两个子区间，我们就把这两个子区间对应的结点作为它的孩子。这样，我们会得到一个树结构。图 2.1 和 图 2.2 展示了这两个结构，它们都可以帮助我们直观地考察数据结构。一般来说，我们会在区间划分结构上思考问题，而分治树结构则只用于提醒我们这是一棵树。

一些资料显示结点是比节点更规范的表述，虽然实际上并没有人在意。在英文中，它们都是 node。



图 2.1: 区间划分

我们立刻可以从图中得到以下的结论：

性质 2.13. 分治树的性质.

- 一层中的区间是不交的。
- 叶子未必具有相同的高度。

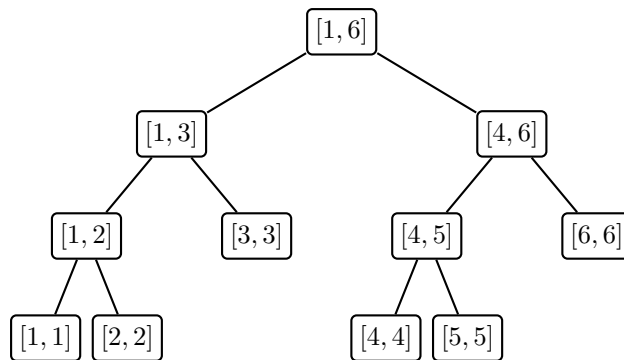


图 2.2: 分治树

- 所有结点的区间长度之和不超过 $n \times \text{高度}$ 。
- 两个叶子的 LCA 对应了它们被划分到两侧的那个分治区间。

练习 2.14. 组合数学复习. 如果允许任意选择分治点（而不是固定为中点），那么有多少种可能的分治树？

下面的练习将询问的复杂度优化到 $O(1)$ 。

练习 2.15. 优化二区间合并的询问复杂度.

1. 假设你有一个 $O(n)$ 预处理、 $O(1)$ 查询 LCA 的算法。在这种情况下，说明询问复杂度可以达到 $O(1)$ 。
2. 即使没有这样的算法，我们也可以实现 $O(1)$ 查询。首先，在序列末尾填充一些 0 来将序列长度补齐为 2^k 。这样我们总是可以假设 n 是 2 的幂，并且不改变预处理复杂度。说明对于这样的 n ，两个叶子的 LCA 可以 $O(1)$ 计算，从而可以做到 $O(1)$ 询问。尝试用代码实现这个算法。
 - [提示一](#)
 - [提示二](#)
 - [提示三](#)

这种算法真的存在，不过也许有点麻烦。

2.3 线段树

在 2.12 中，我们通过预处理的方式回答了前后缀询问。那么，如果我们直接通过递归来回答前后缀询问呢？不妨考虑 $[l, r]$ 的前缀询问 $[l, i]$ 。如果 $i \leq m$ ，那么直接递归 $[l, m]$ ；否则，可以用 $f[l, m] \odot f[m+1, i]$ ，而后者是 $[m+1, r]$ 的一个前缀。我们预处理所有完整区间的 $f[l, r]$ ，询问时通过递归回答。这就得到了线段树。

数据结构 2.16. 线段树.

预处理. 从 $[1, n]$ 开始分治。假设当前的分治区间为 $[l, r]$ ，设其中点为 $m = \lfloor (l+r)/2 \rfloor$ 。如果 $l = r$ ，那么结束；否则，递归处理 $[l, m]$ 和 $[m+1, r]$ ，然后用 $f[l, r] = f[l, m] \odot f[m+1, r]$

计算出本区间的结果。

查询. 对于查询 $[lq, rq]$, 从 $[1, n]$ 开始分治。假设当前的分治区间是 $[l, r]$, 中点为 $m = \lfloor (l+r)/2 \rfloor$ 。如果 $[lq, rq] = [l, r]$, 那么直接返回预处理的 $f[l, r]$; 如果 $[lq, rq]$ 完全在 $[l, m]$ 或者 $[m+1, r]$, 则递归下去处理; 否则, 递归 $\text{query}(l, m, lq, m)$ 和 $\text{query}(m+1, r, m+1, rq)$ 来算出 $f[lq, m]$ 和 $f[m+1, rq]$, 然后返回 $f[lq, m] \odot f[m+1, rq]$ 。

现在来看一下复杂度。预处理的复杂度是 $T(n) = 2T(n/2) + O(1) = O(n)$, 而询问的复杂度初看似乎是 $Q(n) = 2Q(n/2) + O(1) = O(n)$ 。然而, 根据我们先前的分析, 在第一次两侧递归后, 两个询问就都变成了前/后缀询问。而对于前/后缀询问, 只有一侧需要递归计算。所以复杂度是 $Q(n) = Q(n/2) + O(1) = O(\log n)$ 。

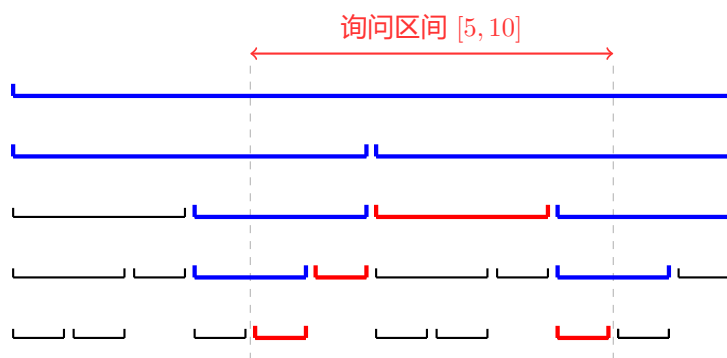


图 2.3: 线段树上定位区间

图 2.3 展示了在线段树上定位 $[5, 10]$ 的情况。红色的区间代表 $[lq, rq] = [l, r]$ 的终止区间, 蓝色的区间代表需要到两侧递归询问的区间。结合上面对询问的分治过程的分析, 我们可以得到以下定理:

定理 2.17. 区间在线段树上的分解. 设线段树树高为 $h \geq 3$ 。对任何询问区间, 其在线段树上的分治过程中至多会访问 $2h - 3$ 个蓝色区间 (需要进一步分治的区间) 和 $2h - 4$ 个红色区间 (可以直接返回的区间)。

评注. 常数 -3 和 -4 其实并不重要。但是系数 2 是重要的。

证明. 先考虑前/后缀询问的情况。除了最后一层, 每层至多产生一个蓝色区间; 除了第一层, 每层至多产生一个红色区间。所以任何前/后缀会访问不超过 $h - 1$ 个蓝色区间和 $h - 1$ 个红色区间。

现在考虑一般的询问区间。在其分解为前后缀询问之前, 每层恰好访问一个蓝色区间。所以最坏情况是在第一层就分解为前后缀询问, 此时总共访问的蓝色区间数量不超过 $1 + 2(h - 2)$, 红色区间数量不超过 $2(h - 2)$ 。□

使用上面的定理时需要知道树高。这是容易看出的:

性质 2.18. 线段树的树高. 按照 $m = \lfloor (l+r)/2 \rfloor$ 划分区间为 $[l, m]$ 和 $[m+1, r]$ 得到的线段树高度为 $h(n) = h(\lceil n/2 \rceil) + 1 = 1 + \lceil \log_2 n \rceil$ 。

评注. 这里暗示了不均匀划分的线段树的存在。虽然不均匀划分一般没什么用，但是会有人拿来出题。

在实现线段树的代码之前，我们需要思考一个问题：如何方便地表示所有预处理的 $f[l, r]$ ？自然的想法是，对分治树的每个结点，在这个结点上记录其对应的区间的 f 。那么问题就转换为了如何存储这棵树。一般的做法当然是指针式的，即每个结点记录其左右儿子结点的编号（或者干脆就是指针）。不过在这里，我们可以类似二叉堆来编号，即让结点 u 的左右儿子分别编号为 $2u$ 和 $2u + 1$ 。图 2.2 中的分治树的结点编号如图 2.4 所示。这种编号方式称为堆式编号。

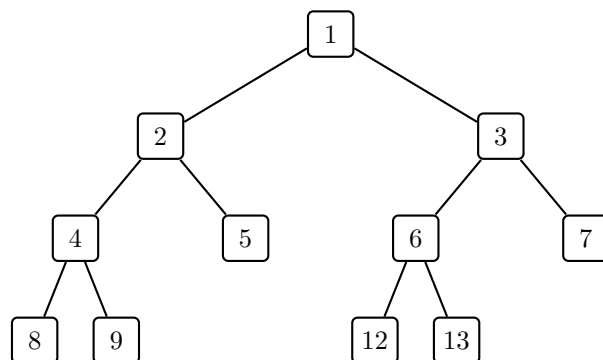


图 2.4: 图 2.2 中分治树的编号

有两个事情需要注意：第一，最后一层也就是叶子层的编号并不是连续的；第二，最大的编号可以达到 $4n - O(1)$ 级别，我们会在练习中仔细计算这个值。

代码 2.4: 线段树

```

1  INFO tree[N<<2];
2  void build(int u, int l, int r) {
3      if (l == r) {
4          tree[u] = a[l];
5          return;
6      }
7      int mid = (l + r) >> 1;
8      build(u << 1, l, mid);
9      build(u << 1 | 1, mid + 1, r);
10     tree[u] = tree[u << 1] + tree[u << 1 | 1];
11 }
12 Info query(int u, int l, int r, int lq, int rq) {
13     if (l==lq && r==rq) {
14         return tree[u];
15     }
16     int mid = (l + r) >> 1;
17     if (rq <= mid) return query(u << 1, l, mid, lq, rq);
18     else if (lq > mid) return query(u << 1 | 1, mid + 1, r, lq, rq);
19     else return query(u<<1,l,mid,lq,mid)+query(u<<1|1,mid+1,r,mid+1,rq);
20 }
  
```

```

21 Info query(int l, int r){
22     return query(1, 1, n, l, r);
23 }

```

评注. 事实上, 线段树可以认为是在描述分治结构。在本讲义中, 我们有时会混用“线段树”和“分治树”, 即将线段树视为一种结构的描述, 而非专门的数据结构。

练习 2.19. 堆式标号线段树的最大标号. 考虑 n 个点的堆式编号线段树, 计算其结点的最大编号。提示

评注. *P7122* 是一个相关题目, 不过要麻烦许多。

我觉得这种包装很无趣。

练习 2.20. 51nod 1792. 定义 $\text{cost}(i, j)$ 为在线段树上分解 $[i, j]$ 时访问的结点数量, 询问 $\sum_{[i,j] \subseteq [l,r]} \text{cost}(i, j)$ 。
类似的题目是洛谷 P5210。

练习 2.21. 线段树的动态开点. 在本练习中, 我们说明可以对长度巨大但是“有意义的位置”很少的序列在合理的时空复杂度内构建线段树。形式化地说, 假设序列长度为 n , 其中只有 m 个位置上的元素被特殊指定, 其余位置上的元素由某种默认模式给出, 如默认置为某个固定元素 x_0 。对于一个不含特殊指定元素的区间, 需要能够在 $O(1)$ 时间内计算这个区间的 $f[l, r]$ 。

1. 堆式编号还能用吗? 应该如何存储树?
2. 在 `build` 中, 如果当前区间 $[l, r]$ 不含特殊指定元素, 那么可以直接计算此区间的 $f[l, r]$ 作为 `tree[u]` 并返回。事实上, 甚至不需要建立这个区间对应的结点。基于这一观察, 计算需要建立的结点的数量级。
3. 如何判断当前区间是否含有特殊指定元素? 提示
4. 修改 `build` 和 `query`, 使得 `build` 的时空复杂度为 $O(m \log n)$, 并且 `query` 能在和原来相同的时间内回答询问。

评注. 可以通过压缩 *Trie* 进一步把空间降为 $O(m)$, 但是一般没有必要。

评注. 在下一章中, 我们会通过单点修改的方式完成相同的目标。

练习 2.22. 分治树的末尾插入. 在本练习中, 我们给出一种从左到右动态构建分治树的算法, 从而让二区间合并和线段树都支持末尾插入。

想象我们有完整的长为 $n = 2^k$ 的序列, 然后在其上建立分治树。而实际上, 在任意时刻, 我们只拥有这个序列的一个前缀。在分治树上, 这个前缀对应“左下角”的一些部分。具体来说, 如果一个结点对应的区间中的元素都已经被插入, 那么我们就称这个结点是完整的。我们的算法思想非常简单: 在末尾插入一个元素后, 把分治树上所有因为这个插入变得完整的结点构建出来。

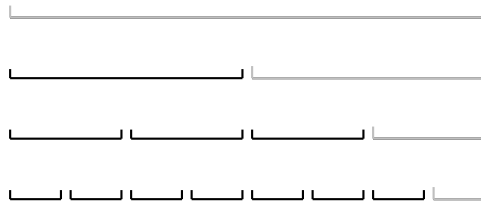


图 2.5: 序列前缀在分治树上

1. 假设目前给出了序列的前 m 个元素。说明我们相当于在长为 $2^{\lceil \log_2 m \rceil}$ 的序列形成的分治树上操作。
2. 末尾插入 a_i 后，哪些结点会变得完整？
3. 说明这个算法在插入 q 次后花费的总时间不超过 $T(2^{\lceil \log_2 q \rceil})$ ，其中 $T(m)$ 是对长为 m 的静态序列构建数据结构所需的时间。所以，在通常的情况下（也就是 $T(m) = m^{O(1)}$ ），每次插入的均摊复杂度不超过 $O(T(q)/q)$ 。分别对于二区间合并和线段树计算这个复杂度。
4. 说明不完整结点永远不会作为 query 中 $[l, r] = [lq, rq]$ 的结点。因此，query 几乎不需要任何改动即可回答询问（只需要修改存储树的方式）。

评注. 另一个理解这个算法的视角是二进制分组：考虑目前序列长度 m 的二进制拆分， $2^{k_1} + 2^{k_2} + \dots + 2^{k_t}$ (k_i 递减)，那么可以视为我们维护了 t 个大小指数递减的分治树，第 i 个分治树对应的区间是 $[2^{k_1} + \dots + 2^{k_{i-1}} + 1, 2^{k_1} + \dots + 2^{k_i}]$ 。在末尾插入时，我们首先新增一棵大小为 1 的线段树，然后暴力做“进位”操作：只要有两棵树的大小相同，那么就将其合并得到两倍大小的线段树。

练习 2.23. 分治树的末尾插入删除。 本练习在 [练习 2.22](#) 的基础上给出一种支持末尾插入删除的算法。这个算法可以配合双端队列实现双端的插入删除。

1. 说明通过反复在某一个特定位置删除插入可以使得 [练习 2.22](#) 中的算法复杂度变劣。如果构建一个结点需要 $\Theta(1)$ 时间，那么均摊复杂度将劣化至 $\Theta(\log q)$ ；如果构建一个结点需要 $\Theta(\text{区间长度})$ 时间，那么均摊复杂度将劣化至 $\Theta(q)$ 。
2. 改变创建非叶子的完整结点的时刻：对于一个非叶结点 u ，当它右边的结点变得完整时才构建 u ；当已构建结点变得不完整时，摧毁此结点。为什么这样就能避免前面的问题？
3. 分析线段树一层内的结点的总构建次数与操作次数 q 的关系。据此说明这个构建算法的均摊复杂度不超过 $O(T(q)/q)$ 。
4. 修改询问函数，使其能正确组合已构建的结点的信息来回答询问，并且复杂度不变。

提示

评注. 也可以通过二进制分组的视角得到类似的算法。我们允许每种大小的分治树有两个，当出现第三个相同大小的分治树时，合并前两个。

评注. *TODO* 插入单元素容易但合并信息复杂。

2.4 多叉分治

在前面，我们用的都是二叉分治，也就是把序列每次分成两半。实际上，我们也可以分成更多叉。比如，我们可以把序列分成 k 个长为 n/k 的子区间，然后递归每个子区间。然后，我们考虑一个询问在这种分治中会被如何划分。有两种情况：完全在某个子区间内，或者跨过两个或更多子区间。后者又进一步分成三个部分：左侧的后缀、右侧的前缀、中间的完整区间。对于前/后缀的询问，我们可以选择像二区间合并一样预处理，也可以选择像线段树一样递归，递归下去会再一次产生一个子区间内的前/后缀询问以及若干个完整区间的询问。这不是分块吗？

我们注意这里的子问题结构。每个子区间当然是子问题，而完整区间的询问实际上也是子问题。这是因为我们可以计算出每个完整区间的 $f[l, r]$ ，然后问题就变成以这些 $f[l, r]$ 为元素的序列上的区间询问。所以，长为 n 的问题会分出来 k 个大小为 n/k 的（块内）子问题以及一个大小为 k 的（块间）子问题。

评注. 如果选择了预处理前后缀询问，那么一个询问就只会在若干次子区间内递归后被拆成前缀 + 若干完整区间 + 后缀的形式直接回答。如果再结合[练习 2.15](#)中的技巧，我们还可以进一步省掉子区间内递归的过程。

我们可以对于两种子问题使用不同的数据结构，并选择是否预处理前后缀询问，然后组合出各种各样的数据结构。在实践中，最常使用的组件是暴力。我们有两种暴力：一种暴力是 $O(n^2)$ 预处理 $O(1)$ 查询的，一种是 $O(n)$ 预处理 $O(n)$ 查询的。我们用 “ $O(f(n)) - O(g(n))$ ” 来表示 $O(f(n))$ 预处理、 $O(g(n))$ 查询。让我们来看一些例子：

例 2.24. 假设初始序列长度为 N 。待处理的子问题的大小则用 n 表示。

- 递归处理大小为 n/k 的块内子问题，使用 $O(k) - O(k)$ 的暴力处理大小为 k 的块间子问题。分别取 $k = 2, \log N$ ，观察得到的数据结构的预处理和查询复杂度。
- 递归处理大小为 n/k 的块内子问题，使用 $O(k^2) - O(1)$ 的暴力处理大小为 k 的块间子问题。取 $k = \sqrt{n}$ ，观察得到的数据结构的预处理和查询复杂度。
- 使用 $O(n/k) - O(n/k)$ 的暴力处理大小为 n/k 的块内子问题，使用线段树处理大小为 k 的块间子问题。取 $k = n/\log n$ ，观察得到的数据结构的时间和空间复杂度（不计算存储序列 a 需要的空间）。另外，这个结构的常数也比直接上线段树更小。这是被称为底层分块的技巧。
- 使用 $O(n/k) - O(n/k)$ 的暴力处理大小为 n/k 的块内子问题，使用 $O(k) - O(k)$ 的暴力处理大小为 k 的块间子问题。取 $k = \sqrt{n}$ ，观察得到的数据结构的预处理和查询复杂度。

2.4.1 应用：线性 RMQ

令 $k = \Theta(n/\log n)$ ，使用任意 $O(k \log k) - O(1)$ 的数据结构处理大小为 k 的块间子问题，使用某种 $O(n/k) - O(1)$ 的数据结构处理大小为 $n/k = \log n$ 的块内子问题。这样，我们就能做到 $O(n) - O(1)$ 的 RMQ。然而，为什么块内的线性复杂度的数据结构可以存在呢？事实上，这非常依赖问题的性质。我们将看到 RMQ 具有这样的性质。

我们先简要介绍“Four Russians”方法的思想。对于 RMQ 来说，我们通过笛卡尔树可以将一般的 RMQ 变成一种特殊的 RMQ—— ± 1 RMQ，也就是在一个相邻项之差总为 ± 1 的序列上做的 RMQ。此时，长为 k 的序列的差分序列只有 2^{k-1} 种，而 RMQ 的结果只由差分序列即可确定。所以，取 $k = \frac{1}{2} \log n$ ，我们可以打表所有长度不超过 k 的差分序列上的所有 RMQ 结果，这部分预处理的复杂度是 $\tilde{O}(2^k) = \tilde{O}(\sqrt{n}) = o(n)$ 的。这样，我们在按 k 分块后就可以对每个块在 $O(k)$ 时间内识别出这个块的差分序列在预处理的表中的位置，查表即可 $O(1)$ 回答询问。一般来说，如果存在一个不太小的函数 $f(n)$ 使得长度为 $f(n)$ 的序列只有 $o(n)$ 种本质不同的结果，并且可以在线性时间内找出一个序列对应的结果，就可以使用这种方法来做到 $O(n) - O(1)$ 。例如，01 序列上的所有询问都满足这个条件。

俗称四毛子。

不过，我们还有一种更简单的方法在 $n \leq w$ 的时候实现 $O(n) - O(1)$ RMQ。参见[练习 1.15](#)。

2.5 * 理论最优的静态区间数据结构

在前面的多叉分治结构中，我们预处理前后缀，在块内子问题上递归，在块间子问题上选择另一个数据结构，并且用 LCA 定位来消去递归到分裂区间的时间。这样，我们可以实现 $T(n) = kT(n/k) + T'(k) + \Theta(n)$ 的预处理和 $Q(n) = \Theta(1) + Q'(k)$ 的询问复杂度。那么二区间合并就是令 $k = 2$ ，此时块间子问题不存在， $T_0(n) = \Theta(n \log n)$ ， $Q_0(n) = \Theta(1)$ 。接下来，我们令 $k = n/\log n$ ，并且让块间子问题使用二区间合并。这样我们得到 $T_1(n) = (n/\log n)T_1(\log n) + T_0(n/\log n) + \Theta(n)$ ， $Q_1(n) = \Theta(1) + Q_0(n)$ 。展开递归式可以发现此时 $T_1(n) = \Theta(n \log^* n)$ ， $Q_1(n) = \Theta(1)$ 。然后，我们再把这个数据结构作为下一个数据结构的块间数据结构，取 $k = \log^* n$ ，就能得到 $T_2(n) = \Theta(n \log^{**} n)$ ， $Q_2(n) = \Theta(1)$ 。如此嵌套下去，我们能得到一个理论最优的数据结构。当然，具体还有许多取整和常数上的细节需要处理，但是大体的思路即是如此。这样，我们可以得到 $O(n\alpha(n)) - O(\alpha(n))$ 的静态区间数据结构。事实上，这个已经是下界了。不过，在实际的数据范围下，这个做法只有理论上的意义。

3 带修区间数据结构

本章中，我们考虑带修区间信息查询，也就是在上一章中的问题的基础上，允许对序列 a 的修改。我们仍然假设需要查询的函数 $f_{[l,r]}(a)$ 具有可分解性，即存在二元运算 $\odot$ 使得 $f_{[l,r]}(a) = f_l(a) \odot \cdots \odot f_r(a)$ 。此外，我们还假设 $f_{[l,r]}(a)$ 只与 $a_{[l,r]}$ 有关。

3.1 单点修改

假设 a_t 发生了变化，那么我们需要在先前的静态数据结构中更新所有包含 t 的预处理区间 $[l, r]$ 的 f 。如果这样的区间太多，我们就无法支持快速的修改。在分治树上， a_t 的变化会导致从根到叶子 $[t, t]$ 这条链上所有的结点的预处理区间的 f 更新。如果预处理区间的数量至少是结点对应的区间长度级别，那么我们就无法支持快速的修改。所以，我们不能预处理前后缀询问。现在的子问题结构如下：

- 对于询问，首先以块内子问题递归若干次，然后分裂成一个块内前缀询问、一个块间询问、一个块内后缀询问。
- 对于前/后缀询问，每层都会分裂出一个块间询问以及一个块内前/后缀询问。
- 对于修改，每次是一个块内子问题，以及更新这个块的信息导致的一个块间子问题。

在一般情况下，这个子问题结构会导致 $T(n) = \min_k T(n/k) + T(k) + 1$ 的复杂度，而这是 $\Theta(\log n)$ 的。所以，线段树已经足够优秀。但有时，修改序列中的一个元素后，我们可以 $O(1)$ 更新完整区间的 f 。此时，可以调整 k 来让修改/查询中的一个复杂度降低、另一个复杂度提高。

评注. 事实上，这里有一个下界。 $[Yao85]$ 证明了代数运算次数的下界满足 $t_q \log(1 + t_u/t_q) = \Omega(\log n)$ ， $[Pătraşcu, Demaine '04]$ 及后续工作则证明了内存访问次数的下界也是 $\Omega(\frac{\delta}{w} \log n)$ ，其中 δ 是每个元素的大小， w 是机器字长。

例 3.1. 多叉线段树. TODO

线段树实现单点修改非常容易。

代码 3.1: 线段树单点修改

```
1 void update(int u, int l, int r, int q, Info val) {
2     if(l == r) {
3         tree[u] = val;
4         return;
5     }
6     int m = (l + r) >> 1;
7     if(q <= m) update(u << 1, l, m, q, val);
8     else update(u << 1 | 1, m + 1, r, q, val);
9     tree[u] = tree[u << 1] + tree[u << 1 | 1];
10 }
```

对于线段树来说，静态查询和单点修改几乎是等难的。我们现在可以完成一大批只需要单点修改的问题。

练习 3.2. 网格图连通性. 给一个 $m \times n$ 的黑白网格， m 很小。支持

- 反转一个格子的颜色
- 查询两个点是否在同一个同色连通块中
- 查询一个子矩形内的同色连通块个数

P4121 P3300 P4246 都是类似的问题。P5897 最短路也可以类似地做（但是转移需要聪明一些）。

3.2 全局修改

在讨论区间修改之前，我们先讨论一个更弱的问题，即全局修改。单点修改总是可以视为赋值。而对于批量修改，我们首先需要对修改进行刻画，以尝试提升计算效率。

定义 3.3. 修改. 假设序列元素都在集合 A 中，所有可能的序列集合记作 A^* 。一个修改是一个 $A^* \rightarrow A^*$ 的变换。所有 $A^* \rightarrow A^*$ 的变换的一个子集 $\mathcal{M}$ 刻画了允许的修改操作。

什么花里胡哨的，这就是函数。

显然，一般的修改可以非常复杂，这样的修改通常也没法处理。下面是一些没那么复杂的例子：

例 3.4.

1. 加法: $\Phi_c(a) = (a_1 + c, \dots, a_n + c)$
2. 和 x 取 $\max$: $\Phi_x(a) = (\max(a_1, x), \dots, \max(a_n, x))$
3. 加等差数列: $\Phi_d(a) = (a_1, a_2 + d, \dots, a_n + (n-1)d)$
4. 翻转: $\Phi(a) = (a_n, \dots, a_1)$
5. 排序: $\Phi(a) = a$ 的升序排列
6. 做前缀和: $\Phi(a) = (a_1, a_1 + a_2, \dots, a_1 + \dots + a_n)$

我们到目前为止讨论的数据结构都是假设序列结构不发生改变，这类数据结构能处理的基本上只有**原地修改**，也就是每个 a_i 的修改只依赖于 a_i 本身，而不依赖于其他位置的元素。形式化地说，这就是 $\Phi(a) = (\phi_1(a_1), \dots, \phi_n(a_n))$ 。原地修改的一个常见的例子是**一致修改**，也就是 $\phi_1 = \dots = \phi_n$ 的情况。

练习 3.5. 上面的例子中，哪些是原地修改甚至一致修改？写出对应的 ϕ_i 。

3.2.1 全局查询

现在，我们考虑一下如何回答全局查询。我们希望全局修改 Φ 可以“整体”地应用到 f 上。具体来说，也就是存在某个 S 上的变换 τ_Φ ，使得 $f(\Phi(a)) = \tau_\Phi(f(a))$ 。这样，我们就需要实际地做修改 $a \leftarrow \Phi(a)$ 再计算 f ，而是直接把 τ_Φ 应用到 $f(a_1, \dots, a_n)$ 上。

紧接着的问题是，如何处理多次修改？我们有

$$\begin{aligned} & f(\Phi_2(\Phi_1(a))) \\ &= \tau_{\Phi_2}(f(\Phi_1(a))) \\ &= \tau_{\Phi_2}(\tau_{\Phi_1}(f(a))) \end{aligned}$$

所以只要逐个应用 τ 即可。

例 3.6. 矩阵乘法. 假设 $\mathbf{a}_i$ 都是 k 维向量，修改是矩阵乘法 $\Phi_{\mathbf{M}}(\mathbf{a}) = (\mathbf{M}\mathbf{a}_1, \dots, \mathbf{M}\mathbf{a}_n)$ ，查询是 $f(\mathbf{a}) = \sum_i \mathbf{a}_i$ 。

3.2.2 区间查询

现在考虑如何回答区间查询。自然的想法是，我们寻找一个能把全局修改“落到”区间上的方法。也就是说，我们寻找 $\tau_{\Phi, [l, r]}$ 使得 $f_{[l, r]}(\Phi(a)) = \tau_{\Phi, [l, r]}(f_{[l, r]}(a))$ 。这样，我们求出 $f_{[l, r]}(a)$ 后应用 $\tau_{\Phi, [l, r]}$ 即可。

然而我们有多次修改，逐个计算 τ 并应用的代价过高。所以，我们希望首先复合出 $\Phi = \Phi_m \circ \dots \circ \Phi_1$ ，然后再计算 $\tau_{\Phi, [l, r]}$ 。这一过程中，我们需要如下性质：

- 修改集合 $\mathcal{M}$ 在复合下的闭包 $\overline{\mathcal{M}}$ 不太大，且 $\overline{\mathcal{M}}$ 中的任意两个修改 Φ_1, Φ_2 都能够快速复合
- 对任何 $\Phi \in \overline{\mathcal{M}}$ ，都可以快速计算 Φ 对 $f_{[l, r]}(a)$ 的影响 $\tau_{\Phi, [l, r]}(f_{[l, r]}(a))$ 。

满足这种性质的 $\overline{\mathcal{M}}$ 中的元素通常也叫做**标记**，它使用较少的信息描述了一个批量修改。

评注. 注意第二个性质与修改 Φ 和信息 f 都相关。有时，无法直接计算 Φ 对 f 的影响，但是当我们把 f 放到一个更大的信息 g 中时， Φ 对 g 的影响也许就会变得容易计算。

一个常见的做法是把 $[l, r]$ 或者 $r - l + 1$ 放到 $f_{[l, r]}(a)$ 中，这样就可以把 $[l, r]$ 从 τ 的参数中去掉，也就是我们只需要考虑 τ_{Φ} 。

评注. 用矩阵乘法来描述修改几乎总是方便的，因为矩阵乘法自动是封闭的。这里矩阵乘法可以是广义运算意义下的，如 $(\max, +)$ 、 $(\max, \min)$ 等。

在实现时，由于修改对应的矩阵乘法可能有特殊的性质，如上三角等。此时，可以只存储矩阵的非零元，并且手工计算这种矩阵乘法来加快效率。例如，计算 2×2 矩阵乘法需要 8 次乘法和 4 次加法，而计算上三角矩阵乘法只需要 4 次乘法和 1 次加法。

例 3.7. 线段树 2. 支持全局加、全局乘，查询区间和。在 $\mathbb{Z}_m$ 或者说模 m 意义下做运算

解答. 这是一个一致修改，我们可以先关注每个元素上的变换。 $\mathcal{M}$ 包含所有的 $x \mapsto x + b$ 以及 $x \mapsto kx$ ，这两种变换的复合是一次函数 $x \mapsto kx + b$ 。由于一次函数的复合还是一次函数，我们就得到了 $\mathcal{M}$ 的闭包 $\overline{\mathcal{M}}$ 的结构：所有逐元素的一次函数变换。

现在，考虑一个 $\overline{\mathcal{M}}$ 中的一个变换 Φ ，假设其执行了逐元素的 $x \mapsto kx + b$ 。我们来寻找

$\tau_{\Phi,[l,r]}$, 这就是

$$\begin{aligned} f_{[l,r]}(\Phi(a)) &= \sum_{i=l}^r k a_i + b \\ &= \left(k \sum_{i=l}^r a_i \right) + (r-l+1)b \\ &= k f_{[l,r]}(a) + (r-l+1)b. \end{aligned}$$

所以我们就得到 $\tau_{\Phi,[l,r]}(f_{[l,r]}(a)) = k f_{[l,r]}(a) + (r-l+1)b$ 。

让我们关注另一个视角。如果我们把区间长度 $r-l+1$ 也加入到 $f_{[l,r]}(a)$ 中, 也就是

$$f_{[l,r]}(a) = \begin{bmatrix} \sum_{i=l}^r a_i \\ r-l+1 \end{bmatrix}, \text{ 那么我们就可以把 } \tau \text{ 写成}$$

$$\tau_{\Phi,[l,r]} f_{[l,r]}(a) = \begin{bmatrix} k & b \\ 0 & 1 \end{bmatrix} f_{[l,r]}(a).$$

这样 τ 和 $[l,r]$ 变得无关了。

例 3.8. 维护两个序列 a 和 b , 支持

- 全局让 a_i 或者 b_i 乘 x 或加 x
- 全局让 a_i 变成 $a_i + b_i x$
- 全局让 b_i 变成 $b_i + a_i x$
- 查询区间 a_i 的和以及 b_i 的和

都在 $\mathbb{Z}_m$ 或者说模 m 意义下做运算。

解答. 直接思考比较困难。然而, 我们很容易用矩阵乘法描述所有的修改, 这是因为所有的修改都是仿射变换:

$$\begin{bmatrix} a_i \\ b_i \end{bmatrix} \mapsto \begin{bmatrix} \alpha & \beta \\ \gamma & \delta \end{bmatrix} \begin{bmatrix} a_i \\ b_i \end{bmatrix} + \begin{bmatrix} u \\ v \end{bmatrix}.$$

那么这就是一个矩阵意义下的一次函数变换。

评注. 现在, 我们可以让每个 a_i 都代表平面上的一个点, 支持全局关于某个点旋转、缩放、平移, 查询区间内的点的坐标和。

练习 3.9. OI 中经常出现的情况是下面这样: 给定序列 a 以及显式或隐式给定的序列 b (如 $b_i = \text{Fib}_i$), 支持全局 a_i 加 x 或乘 x 或让 a_i 变成 $a_i + b_i x$, 查询区间内 a_i 的和。

例 3.10. 维护两个序列 a 和 b , 支持

- 全局让 a_i 或者 b_i 乘 x 或加 x
- 查询区间内 $a_i b_i$ 的和

都在 $\mathbb{Z}_m$ 或者说模 m 意义下做运算。

解答. 根据线段树 2 的例子, 这里的变换还是一次函数。我们考虑如何计算 τ 。有

$$\begin{aligned} f_{[l,r]}(\Phi(a)) &= \sum_{i=l}^r (k_a a_i + d_a)(k_b b_i + d_b) \\ &= k_a k_b \sum_{i=l}^r a_i b_i + k_a d_b \sum_{i=l}^r a_i + d_a k_b \sum_{i=l}^r b_i + (r-l+1)d_a d_b. \end{aligned}$$

所以, 我们需要 $\sum_{i=l}^r a_i b_i$ 、 $\sum_{i=l}^r a_i$ 、 $\sum_{i=l}^r b_i$ 、 $r-l+1$ 这四个信息来计算 $\tau_{\Phi,[l,r]}$ 。那么自然的选择是把这四个信息都加入到 $f_{[l,r]}(a)$ 中。之后我们还需要考虑 $\sum_{i=l}^r a_i$ 、 $\sum_{i=l}^r b_i$ 、 $r-l+1$ 这三个量在一次函数变换下的变化, 这是在线段树 2 中已经解决过的问题。

练习 3.11. 支持以上三个问题中的所有修改和询问。

例 3.12. 维护序列 a , 支持全局加 x , 全局和 x 取 $\max$, 全局赋值为 x , 查询区间 $\max$ 。 x 可以为负值。

解答. 使用 $(\max, +)$ 矩阵乘法来描述修改。 $\max$ 对应标准矩阵乘法中的加法, $+$ 对应标准矩阵乘法中的乘法。注意到修改也是类似仿射变换的形式: $x \mapsto \max(x + \alpha, \beta)$ 。将 a_i 扩充一个 0 来表示常数项, 那么修改就可以用如下矩阵乘法来描述: 加 x 对应 $\begin{bmatrix} a_i \\ 0 \end{bmatrix} \mapsto \begin{bmatrix} x & -\infty \\ -\infty & 0 \end{bmatrix} \begin{bmatrix} a_i \\ 0 \end{bmatrix}$, 和 x 取 $\max$ 对应 $\begin{bmatrix} a_i \\ 0 \end{bmatrix} \mapsto \begin{bmatrix} 0 & x \\ -\infty & -\infty \end{bmatrix} \begin{bmatrix} a_i \\ 0 \end{bmatrix}$, 赋值为 x 对应 $\begin{bmatrix} a_i \\ 0 \end{bmatrix} \mapsto \begin{bmatrix} -\infty & x \\ -\infty & -\infty \end{bmatrix} \begin{bmatrix} a_i \\ 0 \end{bmatrix}$ 。查询区间 $\max$ 就是 $(\max, +)$ 意义下的向量加法。

评注. 通过 *Segment Tree Beats* 技巧, 还可以支持查询区间和。这个技巧依赖和 x 取 $\max$ 这个操作的均摊性质。

在下一节中, 我们将看到以上这些问题都可以简单地扩展到区间修改。现在, 让我们来总结一下标记与信息设计的一般原则。

原则 3.13. 标记与信息的设计原则。

- 首先观察修改在复合运算下的闭包 $\overline{\mathcal{M}}$ 的结构。这通常可以由矩阵刻画。
- 根据 2.6 初步设计分治信息 f 。
- 取一个 $\overline{\mathcal{M}}$ 中的一般修改 Φ , 尝试计算 $\tau_{\Phi,[l,r]}$, 把需要的辅助信息添加进分治信息 f 。
- 重复上一步若干次。如果顺利, 那么可以得到一个 τ 。否则, 可能说明不存在简单的可以快速修改的分治信息。

在实现代码时, 方便的做法是把修改的闭包 $\overline{\mathcal{M}}$ 中的元素封装成一个结构体 **Tag**, 并实现修改的复合以及修改对信息的影响 $\tau_{\Phi}(f)$ 。

代码 3.2: 修改的封装

```
1 struct Tag {
2     // 表示 M 中的一个元素需要的若干变量。我们以线段树 II 为例。
3     long long k, b;
```

这是吉如一 (jiru_2) 发现的技巧, 所以也有吉司机线段树、吉利线段树等名字。正文中这个名字来自番剧 *Angel Beats!*

```

4 // 默认构造函数：对应恒等变换  $x \rightarrow x$ 。如果没有直接的表示，那么可以再加
   // 一个 bool 变量表示。
5 Tag() : k(1), b(0) {}
6 // 构造函数
7 Tag(long long k_, long long b_) : k(k_), b(b_) {}
8
9 // 重载 Tag(Tag) 运算符：实现修改的复合。也可以用其他运算符表示，如 *
   // 。
10 Tag operator()(const Tag &tag)
11 {
12     return Tag(k * tag.k, k * tag.b + b.b);
13 }
14 // 重载 Tag(Info) 运算符：实现  $\tau_{\Phi}(f)$ ，也就是修改如何作用到信息
   // 上的函数。也可以用其他运算符表示，如 *。
15 Info operator()(const Info &info)
16 {
17     return Info(k * info.sum + b * info.len, info.len);
18 }
19 };

```

3.3 区间修改

现在，我们来讨论最后的问题，即区间修改。相比全局修改 Φ ，区间修改 $\Phi_{[l,r]}$ 额外传入了一个区间作为参数。在 OI 的常规语境下，这代表只对 $a_{[l,r]}$ 进行修改。此处仍然只讨论原地修改，常见的情况有以下三类：

- 一致修改： $\Phi_{[l,r]}(a)$ 将每个 $i \in [l, r]$ 的 a_i 替换为 $\phi(a_i)$
- 与相对位置有关的修改： $\Phi_{[l,r]}(a)$ 将每个 $i \in [l, r]$ 的 a_i 替换为 $\phi_{i-l+1}(a_i)$
- 与绝对位置有关的修改： $\Phi_{[l,r]}(a)$ 将每个 $i \in [l, r]$ 的 a_i 替换为 $\phi_i(a_i)$

与询问相同，我们希望修改也具有可分解性。也就是说当对 $[l, r]$ 进行修改时，我们希望能够将修改分解到 $[l, m]$ 和 $[m+1, r]$ 上。形式化地说，对于修改 $\Phi_{[l,r]}$ ，我们希望存在 $\Phi_{[l,m]}$ 和 $\Phi_{[m+1,r]}$ 使得 $f_{[l,r]}(\Phi_{[l,r]}(a)) = f_{[l,m]}(\Phi_{[l,m]}(a)) \odot f_{[m+1,r]}(\Phi_{[m+1,r]}(a))$ 。

例 3.14. 一致修改和与绝对位置有关的修改显然可分解。对于与相对位置有关的修改，通常需要 ϕ_i 容易平移，也就是 $(\phi_{m+1}, \dots, \phi_r)$ 容易用 $(\phi_1, \dots, \phi_{r-m})$ 表示出来。例如， $\phi_i(x) = x + (ki + b)$ 和 $\phi_i(x) = x + kr^i$ 就满足这样的性质。而 $\phi_i(x) = x + b_i$ 则不满足这样的性质。

现在，与静态区间询问类似，我们可以在分治树上递归，把修改 $\Phi_{[L,R]}$ 分解到若干分治区间上，在这些区间上计算 τ 并应用，然后再重算分解过程中影响到的所有区间的 f 。考虑分治树上的一个区间 $[l, r]$ ，有四种情况：

- $[l, r]$ 是 $[L, R]$ 分解到的一个完整区间。那么我们通过 τ 算出了修改后的这个区间的 f 。
- $[l, r]$ 是 $[L, R]$ 分解到的一个完整区间的真子区间。那么虽然我们还没有计算 $\Phi_{[L,R]}$ 对这个区间的 f 的影响，但是我们可以保证这个区间的 f 应用上其分治树上祖先结点存

储的修改后是正确的。

- $[l, r]$ 与 $[L, R]$ 有交但不被包含，也就是说 $[l, r]$ 是 $[L, R]$ 分解过程中的一个区间。此时，我们通过子区间的 f 计算出正确的本区间的 f 。
- $[l, r]$ 与 $[L, R]$ 没有交集，那么这个区间的 f 不受 $\Phi_{[L, R]}$ 的影响。

此时，假设又有一个修改 $\Phi'_{[L', R']}$ ，我们仍然在分治树上做分解。然而，在递归时，有可能当前的分治区间 $[l, r]$ 上已经有先前分解出的修改 $\Phi_{[l, r]}$ 了。有两种情况：

- $[l, r]$ 已经是一个完整区间了。此时，我们直接复合两个修改，也就是令 $\Phi_{[l, r]} \leftarrow \Phi'_{[l, r]} \circ \Phi_{[l, r]}$ 。
- $[l, r]$ 不是一个完整区间。此时，如果直接向下继续分解，那么就会导致新的修改作用在旧的修改之前。由于修改之间往往不交换，这样会导致结果出错。因此，我们需要先把当前区间记录的 $\Phi_{[l, r]}$ 分解到子区间 $[l, m]$ 和 $[m + 1, r]$ 上，然后再继续递归分解 $\Phi'_{[l, r]}$ ，以保证新的修改总是作用在先前的旧修改之后。需要注意的是把 $\Phi_{[l, r]}$ 分解到子区间时也可能出现上一点中的情况。把一个分治区间上的修改分解到子区间上的这个过程叫做 **pushdown**。

也就是说，我们维护了下面这个不变量：对于分治树上的一个区间 $[l, r]$ ，假设其分治树上的祖先自下往上分别存储了修改 $\Phi_1, \dots, \Phi_k$ 。那么在任意时刻，我们需要保证真实的 $f_{[l, r]}(a)$ 等于该结点当前存储的 $\hat{f}_{[l, r]}(a)$ 自下往上地应用其分治树上所有祖先存储的修改得到的结果，也就是

$$f_{[l, r]}(a) = \hat{f}_{[l, r]}(\Phi_k(\dots \Phi_2(\Phi_1(a)))) = (\tau_{\Phi_k} \circ \dots \circ \tau_{\Phi_1})(\hat{f}_{[l, r]}(a)).$$

所以，为了保证修改的顺序正确，我们需要在递归之前对当前结点进行 **pushdown**。

现在来考虑区间询问。我们首先分解到若干完整的分治区间上，然后考虑每个分治区间的结果。根据我们维护的不变量，一个分治区间真实的 $f_{[l, r]}(a)$ 就是其存储的 $\hat{f}_{[l, r]}(a)$ 应用上其祖先存储的所有修改得到的结果。那么一种回答询问的方法就是在递归时记录祖先存储的修改复合的结果，最后再应用到存储的 $\hat{f}_{[l, r]}(a)$ 上。不过，更常见的做法是在递归过程中对访问到的所有结点 **pushdown**。这样做之后，祖先中的就没有修改了。根据我们的循环不变式，此时真实的 $f_{[l, r]}(a)$ 就等于存储的 $\hat{f}_{[l, r]}(a)$ 。

评注. 在递归之前对当前结点 **pushdown** 总是对的。这是最通用和简便的写法。

这样我们就得到了完整的支持区间修改和区间查询的线段树。

代码 3.3: 线段树区间修改

```

1 struct SegTree {
2     int n;
3     vector<Info> tree;
4     vector<Tag> tag; // 维护分治树上每个结点的修改。对于没有修改的结点，
                       tag[u] 就是恒等变换。
5     SegTree(int n_) : n(n_), tree(4 * n_), tag(4 * n_) {}
6
7     void build(int u, int l, int r, const vector<int>&a) {
8         if(l == r) {
9             tree[u] = Info(a[l]);

```



```

10         return;
11     }
12     tag[u] = Tag();
13     int m = (l + r) >> 1;
14     build(u << 1, l, m, a);
15     build(u << 1 | 1, m + 1, r, a);
16     tree[u] = tree[u << 1] + tree[u << 1 | 1];
17 }
18 SegTree(const vector<int>&a) : SegTree(a.size()){
19     build(1, 1, n, a);
20 }
21 // 实现把修改作用到线段树结点上的函数。
22 void apply(int u, const Tag &newtag) {
23     tree[u] = newtag(tree[u]);
24     tag[u] = newtag(tag[u]);
25 }
26 void pushdown(int u, int l, int r) {
27     if(tag[u].isId()) return; // 可以不加，但是加了通常快一些。
28     // 分解 tag[u]。通常就是 tag[u] 本身，此时不需要传入 l,r。
29     int m = (l + r) >> 1;
30     Tag tagL = tag[u], tagR = tag[u];
31     tag[u]=Tag()
32
33     // 将分解后的修改作用到子区间上
34     apply(u << 1, tagL);
35     apply(u << 1 | 1, tagR);
36 }
37 void update(int u, int l, int r, int lq, int rq, const Tag &newtag) {
38     if(l == lq && r == rq) {
39         apply(u, newtag);
40         return;
41     }
42     pushdown(u, l, r);
43     int m = (l + r) >> 1;
44     if(rq <= m) update(u << 1, l, m, lq, rq, newtag);
45     else if(lq > m) update(u << 1 | 1, m + 1, r, lq, rq, newtag);
46     else {
47         update(u << 1, l, m, lq, m, newtag);
48         update(u << 1 | 1, m + 1, r, m + 1, rq, newtag);
49     }
50     tree[u] = tree[u << 1] + tree[u << 1 | 1];
51 }
52 void update(int u, int l, int r, int q, const Info &val) {
53     if(l == r) {
54         tree[u] = val;

```

```

55         return;
56     }
57     pushdown(u, l, r);
58     int m = (l + r) >> 1;
59     if(q <= m) update(u << 1, l, m, q, val);
60     else update(u << 1 | 1, m + 1, r, q, val);
61     tree[u] = tree[u << 1] + tree[u << 1 | 1];
62 }
63 Info query(int u, int l, int r, int lq, int rq) {
64     if(l == lq && r == rq) return tree[u];
65     pushdown(u, l, r);
66     int m = (l + r) >> 1;
67     if(rq <= m) return query(u << 1, l, m, lq, rq);
68     else if(lq > m) return query(u << 1 | 1, m + 1, r, lq, rq);
69     else {
70         return query(u << 1, l, m, lq, m) + query(u << 1 | 1, m + 1,
71             r, m + 1, rq);
72     }
73 }
74 void update(int l, int r, const Tag &newtag) {
75     update(1, 1, n, l, r, newtag);
76 }
77 void update(int q, const Info &val) {
78     update(1, 1, n, q, val);
79 }
80 Info query(int l, int r) {
81     return query(1, 1, n, l, r);
82 }
};

```

通过修改 Info 和 Tag, 我们已经可以完成不计其数的线段树板子题。下面的练习是基本上是最困难的一批, 然而如果你已经完全理解了修改与信息设计的原则, 那么下面的问题基本上都可以一眼看破。

练习 3.15. 括号序列. 维护括号序列, 支持

- 区间赋值
- 反转一个区间内的括号
- 查询区间内左/右括号的数量
- 查询区间内最多有多少个连续的左/右括号
- 查询区间内最长的合法括号子序列的长度
- 查询区间内至少要改变几个括号才能使得区间合法
- 查询区间内有多少个极长连续段
- 查询区间内所有极长连续段长度的最值和 k 次方和

P2572; CF1149C

评注. 利用单侧递归技巧, 我们还能查询区间内最长的合法括号子串的长度, 以及不匹配位置的半群信息。

练习 3.16. 拆位. 维护数列, 支持

- 区间按位异或/按位与/按位或 x
- 查询区间和

练习 3.17. 区间最大子段和. 维护数列, 支持

- 区间赋值
- 区间乘 -1
- 查询区间最大子段和

强于 P4513

评注. 配合模拟费用流可以求出区间最大 k 段子段和。

练习 3.18. 最值. 维护两个数列 a 和 b , 假设 k 是比较小但是不固定的数, 支持

- 两个数列上的区间加, 区间赋值
- 给定长为 k 的序列 $s \in \{a, b\}^k$, 查询区间 $[l, r]$ 的所有长度为 k 的子序列/子段 (设下标为 $i_1 < \dots < i_k$) 中, $\sum_{j=1}^k [s_j]_{i_j}$ 的最值
- 给定长为 k 的序列 s , 查询区间 $[l, r]$ 内是否存在长度为 k 的子段 (设下标为 $i_1 < \dots < i_k$), 使得 $[s_j]_{i_j}$ 单调不降

P7706

练习 3.19. 历史最值. 维护两个数列 a 和 b , 支持

- 两个数列上的区间加、区间赋值、区间和 x 取 $\max$
- 区间内令 $a_i = \max(a_i, b_i + x)$
- 区间内令 $b_i = \max(b_i, a_i + x)$
- 查询两个数列的区间 $\max$

评注. 如果没有第二个操作、第三个操作换成 $b_i = \max(b_i, a_i)$, 那么 b_i 相当于记录了 a_i 的历史最值。

练习 3.20. 数列. 维护数列, 在 $\mathbb{Z}_m$ 也就是 $\text{mod } m$ 下做运算。下面的 k 都是比较小但不固定的数。支持

- 区间加 x , 区间乘 x , 区间赋值为 x
- 查询区间内所有数的 k 次方和
- 查询区间内所有相邻 k 个数的乘积的和
- 查询区间内所有 k 元组乘积的和

P4247, P5142

练习 3.21. 哈希. 维护数列, 固定 B 和 p , 支持

- 区间加, 区间乘, 区间赋值
- 计算区间的 B 进制哈希值 $\sum_{i=l}^r a_i B^{r-i} \bmod p$

练习 3.22. gcd. 维护数列, 支持

- 区间加, 区间赋值
- 查询区间内所有数的 gcd

提示

评注. 注意无论按照何种顺序, 顺次计算 n 个数的 gcd 的复杂度都是 $O(n + \log V)$ 。

练习 3.23. 小值域. 维护数列 a 和另一个分治信息序列 b , a 在 $\mathbb{Z}_k$ 也就是模 k 下做运算且 k 很小, 支持

- 给定 $x, y \in \mathbb{Z}_k$ 把区间内所有 $a_i = x$ 的 a_i 变成 y
- 查询区间内 a 的中位数和众数
- 给一个集合 $S \subseteq \mathbb{Z}_k$, 查询区间内最短的子段, 使得所有 $x \in S$ 都在这个子段内出现
- 给一个集合 $S \subseteq \mathbb{Z}_k$, 把区间内所有 $a_i \in S$ 的位置拉出来形成一个子序列, 然后询问这个子序列的 b 的合并结果

练习 3.24. 加矩阵幂. 固定矩阵 M , 维护向量列 a , 支持

- 给定向量 v 和区间 $[l, r]$, 对所有 $i \in [l, r]$, 令 $a_i \leftarrow a_i + M^{i-l} v$
- 区间加, 区间乘任意矩阵, 区间赋值
- 查询区间和

评注. 这个包括了区间加线性递推的情况, 而线性递推又包括了区间加等差和等比数列的情况。

3.4 特殊情况的优化

3.4.1 标记永久化

需要 `pushdown` 的原因是标记之间不交换。然而标记经常是可交换的, 比如只有区间加、只有区间乘、只有区间和 x 取 $\max$ 等等。对于这种情况, 我们就不需要 `pushdown` 了。在查询时, 我们通过递归计算出没有作用过当前结点以及祖先的修改的信息 $\hat{f}_{[lq, rq]}(a)$, 然后再应用当前结点的修改 $\Phi_{[l, r]}$ 即可。

例 3.25. 矩形面积并. 给定平面上的若干矩形, 我们要计算它们的并的面积。这个问题的做法是扫描线, 也就是考虑每个横坐标段 (l, r) , 满足所有矩形的横坐标区间要么与 (l, r) 不交, 要么完全包含 (l, r) 。这样的段可以由所有矩形的横坐标形成的 $2n$ 个点切分出来。之后, 我们只需要考虑每个段的纵坐标区间并的长度, 而这是一个一维的问题。从一个段移动到它的下一个段的时候, 会加入或者离开一个纵坐标区间。以上这些操作形成了下面的子问题:

维护数轴，支持

- 插入一条线段
- 删除一条线段，保证曾经插入过，并且还没有被删除
- 查询当前所有线段的并的长度

练习 3.26. 上面的问题可以泛化成：维护数列，支持

- 区间加 1，区间减 1，保证任意时刻所有位置非负。
- 查询有多少个位置大于 0。

注意这个问题不能用上一题中的标记永久化方法解决。提示

评注. 如果没有所有位置始终非负的假设，这个问题有矩阵乘法归约，无法做到 $\text{polylog}n$ 。在有了这个假设时，这是个比较常见的技巧。

3.4.2 增量修改与前缀查询：树状数组

当查询都是前缀形式的时候，可以发现线段树的所有左儿子都是无用的。此时，可以直接将所有的左子结点删去，把剩余的结点重连成树。这就得到了树状数组。不过，实际使用时，另一种理解可能会更加方便。定义 $\text{lowbit}(i)$ 为 i 的二进制表示中最低位的 1 所代表的数，如 $\text{lowbit}(20) = \text{lowbit}((10100)_2) = (100)_2 = 4$ 。那么树状数组就是维护一个 n 大小的数组 c ，其中 c_i 存储了 $f_{(i-\text{lowbit}(i), i]}(a)$ 。这样，查询前缀 $f_{[1, r]}(a)$ 就是把 $[1, r]$ 拆成若干 $(x - \text{lowbit}(x), x]$ 然后合并。对于增量修改（修改 a_i 后可以直接算出此修改对包含 i 的区间信息 $f_{[l, r]}(a)$ 的影响），可以找出所有包含 i 的 $(x - \text{lowbit}(x), x]$ ，然后修改对应的 c_x 。

数据结构 3.27. 树状数组.

预处理. 初始令所有 $c_i = f_i(a)$ 。从 0 开始枚举所有 k ，对每个 2^{k+1} 的倍数 i ，令 $c_i = c_{i-2^k} \odot c_i$ 。

查询. 初始令 $s = 0$ 。对于查询 $[1, r]$ ，不断地执行 $s \leftarrow c_r \odot s$ ， $r \leftarrow r - \text{lowbit}(r)$ ，直到 $r = 0$ 。返回 s 。

修改. 对于修改 $a'_x = \varphi(a_x)$ ，假设存在 $\tau_{\varphi, [l, r]}$ 使得对任何包含 x 的区间 $[l, r]$ ，都有 $f_{[l, r]}(a') = \tau_{\varphi, [l, r]}(f_{[l, r]}(a))$ 。不断地令 $c_x \leftarrow \tau_{\varphi, (x-\text{lowbit}(x), x]}(c_x)$ ， $x \leftarrow x + \text{lowbit}(x)$ ，直到 $x > n$ 。

不难发现预处理的复杂度为 $O(n)$ ，而修改查询的复杂度都是 $O(\log n)$ 。实现代码时，有一个细节需要处理——如何计算 $\text{lowbit}(x)$ ？可以注意到这等于 $x \& (\sim(x-1))$ 。此外，根据整数的表示法，我们还有 $\sim(x-1) = -x$ 。在代码中，通常会用 $x \& (-x)$ 来实现 lowbit 。

代码 3.4: 树状数组

```
1 Info s[N];
2 // 每一层分治对应一层数组
```

```

3 void build(int n) {
4     for(int i=1;i<=n;i++)
5         s[i]=Info(a[i]);
6     for(int k=1;k<=n;k<=<1)
7         for(int i=k<<1;i<=n;i+=(k<<1))
8             s[i]=s[i-k]+s[i];
9 }
10 Info query(int r) {
11     Info ans;
12     for(;r;r-=r&(-r))
13         ans=s[r]+ans;
14     return ans;
15 }
16 void upd(const Modify &phi, int x, Info &info) {
17     // 应用 phi 对区间 (x-lowbit(x),x] 的信息 info 的影响)
18 }
19 void update(int x, const Modify &phi) {
20     for(;x<=n;x+=x&(-x))
21         upd(phi,x,s[x]);
22 }

```

练习 3.28. 树状数组的常见应用.

- 单点加, 单点赋值, 查询区间和
- 单点加非负整数, 单点和 x 取 $\max$, 查询前缀 $\max$ 和后缀 $\max$

练习 3.29. 假设修改和查询的位置在 $[1, n]$ 中均匀随机, n 是 2 的幂。说明查询和修改的期望循环次数是 $\frac{1}{2} \log_2 n$ 。

练习 3.30. 树状数组的区间查询与任意单点修改。说明树状数组可以在 $O(\log^2 n)$ 时间内查询区间信息。对于一般的单点修改, 说明树状数组可以在 $O(\log^2 n)$ 内实现。

3.4.3 可差分的修改

像区间加、区间异或这种修改可以通过差分来变成单点修改, 这样就能通过提高查询的代价来简化修改。由于区间修改通常是麻烦的, 能差分的问题总是应该差分。之后, 维护的信息还常常是可减的, 此时可以通过树状数组大大减小常数。

例 3.31. 假设 p 是质数。在 $\mathbb{F}_p$ 或者说模 p 意义下, 乘法不是直接可差分的修改, 因为存在乘 0 的情况。然而如果我们把 0 视为变量 x 并且在 $\mathbb{F}_p(x)$ 下做运算, 那么乘法就是可差分的了。具体来说, 我们把每个元素视为 ax^b , 那么区间乘除 0 就变成了 b 的区间加减。维护区间乘积也同理。需要注意这个做法只适用于只有乘法的情况。

例 3.32. 支持

- 区间加
- 查询区间和

差分后使用树状数组完成。

4 线段树的进阶操作

4.1 zkw 线段树

zkw 线段树放弃了线段树的自上而下分治结构，而是改为自下而上合并。初始所有的叶子结点都是一个单元素区间 $[i, i]$ ，我们每次把所有的结点两个两个地合并成更大的区间，两端可能剩下的孤点不管。这样得到新的一些长为 2 的区间，再继续做这个合并过程，直到只剩一个结点。加上每层得到的孤点，我们会形成一个森林结构。

zkw 线段树的精髓在于标号。叶子 $[i, i]$ 的标号为 $n + i - 1$ ，一个结点 u 的父亲标号为 $\lfloor n/2 \rfloor$ 。通过这样的标号方式，我们可以直接获得叶子结点的标号。考虑如何在 zkw 线段树上分解区间。我们用左闭右开区间 $[l, r)$ 来避免一些特判。从叶子开始，我们有左右两个区间结点 $u = l + n - 1, v = r + n$ 。在 v 是一个右儿子时，记 v 的左邻居为 v' 。如果 u 是一个右儿子，那么 u 会作为一个完整区间出现；否则， u 会被其某个祖先作为完整区间包含。也就是说， u 是分解出的最终区间当且仅当 u 是一个右儿子。同理， v' 是分解出的最终区间当且仅当 v 是一个右儿子。确定完 u, v' 是否是分解出的最终区间后，我们就把 u, v 往上跳一层，递归寻找上面的结点，直到 u, v 相同为止。这样我们就找到了所有分解出的最终区间。在有区间修改时，我们还需要知道所有的非最终区间来 **pushdown**。非最终区间分为两部分：一部分是最终区间的父亲，另一部分是 u, v 相同后到根路径上的结点。

代码 4.1: zkw 线段树

```
1 struct SegTreeZKW
2 {
3     vector<Info> info;
4     vector<Tag> tag;
5     int n;
6     SegTreeZKW(int n): n(n)
7     {
8         info.resize(2*n+2);
9         tag.resize(2*n+2);
10    }
11    template<typename SeqType>
12    SegTreeZKW(const vector<SeqType>& a): SegTreeZKW(a.size()-1)
13    {
14        build(a);
15    }
16    void apply(int u, const Tag& newtag)
17    {
18        info[u] = newtag(info[u]);
19        tag[u] = newtag(tag[u]);
20    }
21    void pushdown(int u)
22    {
```

```

23     if (!tag[u].isId())
24     {
25         apply(u << 1, tag[u]);
26         apply(u << 1 | 1, tag[u]);
27         tag[u] = Tag();
28     }
29 }
30 template<typename SeqType>
31 void build(const vector<SeqType>& a)
32 {
33     for(int i=1;i<=n;i++) info[n + i] = Info(a[i]);
34     for(int i=n-1;i>=1;i--)
35         info[i] = info[i << 1] + info[i << 1 | 1];
36 }
37 Info query(int lq, int rq)
38 {
39     lq += n; rq += n+1;
40     for(int i=__lg(n)+1;i>=1;i--)
41     {
42         pushdown(lq >> i);
43         if ((rq >> i) != (lq >> i))
44             pushdown(rq >> i);
45     }
46     Info resl, resr;
47     while(lq < rq)
48     {
49         if (lq & 1) resl = resl + info[lq++];
50         if (rq & 1) resr = info[--rq] + resr;
51         lq >>= 1; rq >>= 1;
52     }
53     return resl+resr;
54 }
55 void update(int lq, int rq, const Tag& newtag)
56 {
57     lq += n; rq += n+1;
58     for(int i=__lg(n)+1;i>=1;i--)
59     {
60         pushdown(lq >> i);
61         if ((rq >> i) != (lq >> i))
62             pushdown(rq >> i);
63     }
64     int u=0, v=0;
65     while(lq < rq)
66     {
67         if (lq & 1)

```

```

68         {
69             u=lq;
70             apply(lq++, newtag);
71         }
72         if (rq & 1)
73         {
74             v=rq;
75             apply(--rq, newtag);
76         }
77         do{u>>=1; info[u]=info[u << 1] + info[u << 1 | 1];}while(lq==
            rq && u);
78         do{v>>=1; info[v]=info[v << 1] + info[v << 1 | 1];}while(lq==
            rq && v);
79         lq >>= 1; rq >>= 1;
80     }
81 }
82 };

```

注意到这种方式的空间从 $4n$ 降低到了 $2n$ 。在时间上，zkw 线段树只能节省递归的开销，而不能降低信息或标记合并的代价。zkw 线段树节省的递归开销大约相当于维护四到八个 `unsigned` 的和。所以，当信息或标记复杂时，这个性能提升就变得较为微弱。如果实现得不好，可能还会有额外的信息或标记合并开销。

4.2 动态开点

线段树的单点修改会访问至多 $\lceil \log_2 n \rceil + 1$ 个结点，区间修改会访问至多 $4\lceil \log_2 n \rceil - 3$ 个结点。没有被修改过的结点可以不显式建出结点，而是按照某种“默认模式”处理。

代码 4.2: 动态开点线段树

```

1 // 不需要 build 了，可以使用 update 来替代 build
2 // 假设不存在的结点编号为 0，使用 newNode(l,r) 来创建新结点，初始化为 [l,
   //  r] 的默认信息
3 // 大多数情况下，默认信息都是 0，此时 newNode 和 pushdown 都不需要传入 l,
   //  r。
4 int newNode(int l, int r)
5 {
6     ++nodeCnt;
7     ch[nodeCnt][0] = ch[nodeCnt][1] = 0;
8     info[nodeCnt] = defaultInfo(l, r);
9     tag[nodeCnt] = Tag();
10 }
11 void pushdown(int u, int l, int r)
12 {
13     int mid = (l + r) >> 1;
14     if(!tag[u].isId())

```

```

15     {
16         if(!ch[u][0]) ch[u][0]=newNode(l, mid);
17         if(!ch[u][1]) ch[u][1]=newNode(mid + 1, r);
18         apply(ch[u][0], tag[u]);
19         apply(ch[u][1], tag[u]);
20         tag[u] = Tag();
21     }
22 }
23 Info query(int u, int l, int r, int lq, int rq)
24 {
25     if(!u) return defaultInfo(lq,rq);
26     if (lq == l && r == rq)
27         return info[u];
28     int mid = (l + r) >> 1;
29     pushdown(u, l, r);
30     if (rq <= mid)
31         return query(ch[u][0], l, mid, lq, rq);
32     else if(lq > mid)
33         return query(ch[u][1], mid + 1, r, lq, rq);
34     else
35         return query(ch[u][0], l, mid, lq, mid) + query(ch[u][1], mid +
36             1, r, mid + 1, rq);
37 }
38 void update(int &u, int l, int r, int lq, int rq, const Tag& newtag)
39 {
40     if(!u) u=newNode(l,r);
41     if(lq == l && r == rq)
42     {
43         apply(u, newtag);
44         return;
45     }
46     int mid = (l + r) >> 1;
47     pushdown(u, l, r);
48     if (rq <= mid)
49         update(ch[u][0], l, mid, lq, rq, newtag);
50     else if (lq > mid)
51         update(ch[u][1], mid + 1, r, lq, rq, newtag);
52     else
53     {
54         update(ch[u][0], l, mid, lq, mid, newtag);
55         update(ch[u][1], mid + 1, r, mid + 1, rq, newtag);
56     }
57     info[u] = (ch[u][0] ? info[ch[u][0]] : defaultInfo(l,mid)) + (ch[u]
58         [1] ? info[ch[u][1]] : defaultInfo(mid+1,r));
59     // 如果 defaultInfo 的代价比较大, 那么也可以选择在这里把不存在的子节

```

点建出来。这样做会让空间变为两倍。

58 }

例 4.1. CF803G Periodic RMQ Problem. 维护序列 a ，初始为某个序列 b 重复 k 次得到的序列。支持区间赋值、查询区间最值。

4.3 线段树上二分

有时，我们需要支持查询一个区间内满足某个条件的最左或最右位置，如查询区间内第一个前缀 $\max$ 大于等于某个值的位置。此时，直接的想法是套上一层二分，内部用线段树来判断。然而，我们可以利用线段树的结构，直接在线段树上进行二分查找，从而省掉一个 $\log$ 。从理论上来说，我们可以先把区间在线段树上分解为不超过 $2\log n$ 个区间，然后从左到右遍历找到答案所在的完整区间，然后递归进那个完整区间分治查找。不过，实现上要容易得多：

代码 4.3: 线段树上二分

```

1 //以查询区间 [l,r] 内最左的满足  $s[l,i] \geq x$  的  $i$  的位置为例
2 int query(int u, int l, int r, int lq, int rq, Info& preinfo, int x)
3 {
4     if(l==lq&&r==rq)
5     {
6         if(info[u].s<x)
7         {
8             preinfo = preinfo + info[u];
9             return -1;
10        }
11        // 否则，说明答案就在此区间内
12    }
13    if(l==r)
14        return l;
15    pushdown(u);
16    int mid=(l+r)>>1;
17    if(rq<=mid)
18        return query(u<<1, l, mid, lq, rq, preinfo, x);
19    else if(lq>mid)
20        return query(u<<1|1, mid+1, r, lq, rq, preinfo, x);
21    else
22    {
23        int ans=query(u<<1, l, mid, lq, mid, preinfo, x);
24        if(ans!=-1) return ans;
25        else return query(u<<1|1, mid+1, r, mid+1, rq, preinfo, x);
26    }
27 }
28 int query(int lq, int rq, int x)
29 {

```

```
30     return query(1,1,n,lq,rq,Info(),x);  
31 }
```

练习 4.2. 树状数组上二分. 在树状数组上二分来实现：查询序列中最靠左的满足某个条件的位置。

练习 4.3. 权值线段树.

- 假设值域为 V ，通过维护每个值的出现次数来在 $O(\log V)$ 的时间和 $O((n+q) \log V)$ 的空间内实现普通平衡树的所有功能。
- 在 $V = O(n)$ 时，使用树状数组来实现。

4.4 线段树分裂与合并

4.5 单侧递归

4.6 李超线段树

4.7 Segment Tree Beats

4.8 KTT

5 扫描线与几何模型

通过扫描问题的某一维度并考虑两个相邻状态之间的差，我们往往可以将一个多维范围查询问题变为少一维的范围修改查询问题。在本章中，我们尝试用一些例子让大家感受**维度**，并给出十分常用的**平面矩形建模**。

本章的大部分问题都可以直接通过可持久化数据结构实现强制在线，我们在此仅考虑离线情况。

5.1 平面数点问题

很多时候我们会遇到由不等式框定的查询范围，如 $l \leq x \leq r$ 。此时，一个可能有利于直观的办法是将不等式的限制转到平面坐标系中。每一个不等号都对应一个半平面，多个不等式的交集就对应一个平面图形。我们可以把每个元素看成平面上的一个点，那么查询就变成了数点问题：查询范围内所有点的信息和。

5.1.1 可减信息

对于可减的情况，可以通过容斥把 $\geq$ 变成 $\leq$ 。所以每个维度只需要处理单侧的限制。

例 5.1. 矩形数点. 给定一些 (x_i, y_i, w_i) ，多次查询

$$\sum_i [l_x \leq x_i \leq r_x][l_y \leq y_i \leq r_y] w_i.$$

这对应于：给定平面上 n 个带权点，多次查询一个矩形内的点权和。

通过坐标变换（或者说变量代换），可以把平行四边形变成矩形。

例 5.2. 平行四边形数点. 给定一些 (x_i, y_i, w_i) ，多次查询

$$\sum_i [l_x \leq x_i \leq r_x][y_i - x_i \in [l_d, r_d]] w_i.$$

这对应于：给定平面上 n 个带权点，多次查询一个方向固定的平行四边形内的点权和。

评注. 在这里，“方向固定”指的是围成形状的这些半平面的边界线的斜率是固定的。

通过在无限远处差分，可以把直角三角形变成两种四边形的差。

例 5.3. 三角形数点. 给定一些 (x_i, y_i, w_i) ，多次查询

$$\sum_i [l_x \leq x_i \leq r_x][y_i - x_i \in [l_d, r_d]][y_i + x_i \in [l_s, r_s]] w_i.$$

这对应于：给定平面上 n 个带权点，多次查询一个等腰直角三角形内的点权和。

练习 5.4. 把这个问题旋转 45 度。事实上，对于任何方向固定的三角形都可以做。

由于多边形总是可以切分成三角形，我们在理论上就可以做任意的方向固定的不规则图形的数点了。

例 5.5. 梯形数点. 给定一些 (x_i, y_i, w_i) ，多次查询

$$\sum_i [l_x \leq x_i \leq r_x][y_i - x_i \in [l_d, r_d]][y_i + x_i \in [l_s, r_s]]w_i.$$

这对应于：给定平面上 n 个带权点，多次查询一个等腰梯形或者直角梯形内的点权和。

练习 5.6. * 半平面交数点. 给定一些 (x_i, y_i, w_i) ，多次查询

$$\sum_i [x_i \in [l_x, r_x]][y_i \in [l_y, r_y]][x_i - y_i \in [l_d, r_d]][x_i + y_i \in [l_s, r_s]]w_i.$$

这对应于：给定平面上 n 个带权点，多次查询一个由一些半平面交出的一个“规则”多边形内的点权和。

5.1.2 不可减信息

把以上问题中的求和换成 $\max$ 。此时，无法通过容斥把 $\geq$ 变成 $\leq$ ，而是需要通过分治来实现。在此时，问题基本上就被一般化为 d 维正交范围查询问题。通常来说， d 个半平面的限制会带来 $\tilde{O}(\log^{d-1} n)$ 的复杂度。我们将在下一章中看到常见的做法。

5.1.3 转置

对于数点问题，我们可以有另一个理解。假设我们给点用 $1, \dots, n$ 编号，询问用 $1, \dots, q$ 编号。考虑矩形 $\mathbf{A}_{ij} = [\text{点 } j \text{ 在询问 } i \text{ 的范围内}]$ ，那么我们相当于在计算矩阵乘法 $\mathbf{A} \cdot \mathbf{w}$ ，其中 $\mathbf{w}$ 是点的权值向量。此问题的转置为：假设每个询问 i 都有一个权值 u_i ，我们想要计算 $\mathbf{A}^T \mathbf{u}$ ，即对每个 i 计算 $\sum_j [\text{点 } j \text{ 在询问 } i \text{ 的范围内}]u_i$ 。这相当于：平面上有一些点，每个询问 i 需要对其范围内的点的答案加上 u_i ，最后计算每个点的结果。这个也称为对偶数点问题。

什么是对偶？

通常来说，转置问题可以通过转置原理做到与原问题相同的复杂度。不过，在这里，我们只需要正常的思考即可得到合理的做法。

例 5.7. 对偶二维数点. 给定平面上 n 个带权矩形，多次查询包含某个点的矩形的权值和。

5.2 矩形并问题

例 5.8. 矩形面积并. 给定平面上的若干矩形，求其并的面积

为什么不叫矩形并面积？

练习 5.9. 至少被 k 个矩形覆盖的面积。

练习 5.10. 矩形并的周长.

5.3 常见平面建模

有许多东西都可以被视为维度：序列下标、权值、时间等等。我们可以把它们中的一个或几个转化为平面坐标系中的维度，从而把问题转化为数点问题来解决。

例 5.11. 序列 $\times$ 值域. 给一个序列 a ，每次给定 l, r, x, y ，查询有多少个 $i \in [l, r]$ 满足 $a[i] \in [x, y]$ 。

例 5.12. 多次询问：如果把 $a[l, r]$ 加一，那么序列中有多少个不同的值

例 5.13. 序列 $\times$ 序列. 给两棵树 T_1 和 T_2 ，每次询问有多少个点在 T_1 中位于 u 的子树内且在 T_2 中位于 v 的子树内。

例 5.14. 区间包含关系. 初始给定 n 个区间 $[l_i, r_i]$ 。每次给出 $[L, R]$ ，询问

- 有多少个区间包含 $[L, R]$?
- 有多少个区间与 $[L, R]$ 相交?
- 有多少个区间被 $[L, R]$ 包含?

例 5.15. 给一棵树，点从 1 到 n 编号，边从 1 到 $n-1$ 编号。每次询问：只保留编号在 $[l, r]$ 内的点和编号在 $[l, r]$ 内的边时，这些点的导出子图有多少个连通块。提示

例 5.16. NOIP2025 序列询问. 每次询问给定 $[L, R]$ ，对每个 i 求出所有包含 i 且区间长度在 $[L, R]$ 内的区间的和的最大值

例 5.17. 序列 $\times$ 时间. 给一个序列 a ，支持单点加、查询区间和。将此问题视为二维数点。

5.4 区间问题

我们经常通过扫描线来处理区间询问。此时，通常有两种理解：第一种是在扫描 r 的过程中维护所有 l 的答案；另一种是把 l, r 视为两个维度，则所有可能的区间构成平面上的一个三角形。虽然通常来说前一种理解更常用，但也不要忘记后者。

5.4.1 数颜色类问题

有一些问题是查询区间内出现过的值的集合的一些信息。我们展示一些常见的例子和处理手法。

例 5.18. 区间数颜色 给定两个数列 a, b ，查询所有 $a[l, r]$ 中出现过的值 w 的 b_w 之和。

做法一. 考虑一种拆贡献的方式: 一个 $[l, r]$ 内的位置有贡献当且仅当其上一次出现的位置在 $[l, r]$ 之外。这样, 我们预处理出每个值的上一次出现位置 p_i , 则询问就变成 $i \in [l, r], p_i < l$ 的二维数点。

做法二. 也可以考虑另一种拆贡献的方式: 枚举所有权值 w , 当 $a[l, r]$ 中包含 w 时产生贡献。那么考虑一个值 w 的所有出现位置, 当 $[l, r]$ 包含其中某个位置时, 在平面上的 (l, r) 位置产生 b_w 的贡献。注意到如果做一步容斥, 即考虑**不包含** w 的任意一次出现的区间, 则所有这样的区间在平面上可以由 出现次数 + 1 个三角形表达。而由于我们不会询问 $l > r$, 也可以把三角形换成矩形。这样, 问题就变成: 有 $O(n)$ 个矩形加, 最后查询单点值, 即对偶二维数点。

做法三. r 从 1 扫描到 n , 维护每个 l 的答案。

假设 $a[i]$ 的上一次出现位置是 j , 那么 $(j, i]$ 的答案加一, 其他位置不变。

评注. 在下一章中我们来处理带修的情况。

练习 5.19. 区间内只出现一次的数的和。

练习 5.20. 区间内出现恰好 k 次的数的和。

练习 5.21. CF1000F. 找出区间内任何一个只出现了一次的数, 或者报告不存在

练习 5.22. 区间内出现偶数次的数的异或和

例 5.23. 给定两个数列 a, b , 每次给定 $[l, r]$, 查询: $\sum_{w \in a[l, r]} \max\{b_i : a_i = w\}$, 也就是对每个在 $a[l, r]$ 中出现过的值 w , 计算其所有出现位置中 b 的 $\max$, 再求和的 b_w 之和。

练习 5.24. CF1422F. 区间 lcm (对常数模数取模), 值域和 n, q 同阶。

评注. $P5655$ 是值域非常大但 n, q 比较小的情况。

练习 5.25. CF522D. 查询区间内等值对的最小距离

5.4.2 mex 类问题

集合 S 的 mex 定义为不在 S 中的最小非负整数。我们也可以通过扫描线来处理一些 mex 类问题。

例 5.26. 区间 mex. 查询区间内最小的没有出现过的自然数 (mex)。

做法一. 在扫描线的过程中, 维护每个值最靠右的出现。查询时在线段树上二分。

做法二. 在扫描线的过程中直接维护每个 l 的答案 m_l , 则 m_l 构成若干单调不降的连续段。假设 r 右移时遇到了元素 x , 则只有对于 $m_l = x$ 的那些 l , 答案会发生变化。假设这些 l 对应的连续段为 L, R , 我们可以 $O(\text{新的段数})$ 更新这一段的 m 。首先使用做法一求出 m_R , 然

后查找 m_R 在原数组中上一次出现的位置 t ，则 $(t, R]$ 这一段会更新为 m_R ，然后令 $R \leftarrow t$ 继续做，直到 $t < L$ 。

事实上，上面的做法二可以导出区间 mex 的一个重要性质。

定义 5.27. 极小 mex 区间. 如果区间 $[l, r]$ 满足其任何真子区间的 mex 都不等于 $[l, r]$ 的 mex，则称 $[l, r]$ 是一个极小 mex 区间。

定理 5.28. 极小 mex 区间的结构. 极小 mex 区间即为做法二中的所有满足 $t \geq L$ 的 $[R, r]$ 。

练习 5.29. 证明以下性质：

- 除了单点以外，极小 mex 区间至多有 $n + \min(V, n)$ 个。
- 极小 mex 区间之间没有包含关系
- 极小 mex 区间 $[l, r]$ 的 mex 为 $\max(a_l, a_r) + 1$ ，且 a_l, a_r 在 $[l, r]$ 内都只出现过一次。
- 一个区间 $[l, r]$ 的 mex 为被 $[l, r]$ 包含的极小 mex 区间中 mex 的最大值
- 假设极小 mex 区间 $[l, r]$ 的 mex 为 k ，则所有包含 $[l, r]$ 的 mex 为 k 的区间都是 (L, R) 的一个子区间，其中 L, R 分别为 k 在 $[l, r]$ 左/右边第一次出现的位置
- mex 为 k 的区间在平面上可以表示为 $O(\text{cnt}_k)$ 个矩形，其中 cnt_k 是 mex 为 k 的极小 mex 区间的个数

有人想算算精确上界吗？

练习 5.30. 对每个 k ，询有多少个区间的 mex $> k$ 。

做法一. 对每个右端点 r ，维护 $l_k(r)$ 为可能的最靠右的左端点，则答案即 $\sum_r r - l_k(r) + 1$ 。当 k 增加时，相当于 $O(\text{出现次数})$ 次区间和 x 取 min。注意 $l_k(r)$ 关于 r 单调不降，故区间取 min 可以二分出发生变化的位置后变成区间赋值。还需要维护全局和。ODT 或者线段树处理都可以。

做法二. 计算出所有极小 mex 区间，然后对每个 k 使用矩形面积并计算出 mex 为 k 的区间数量。

练习 5.31. 求所有区间的 mex 之和。

5.4.3 子区间类问题

有一些问题查询区间内所有子区间的信息和。此时，我们可以扫描 r ，然后维护每个 l 的答案的历史和。

例 5.32. 全局固定 k 。每次询问区间内有多少个子区间满足 mex $> k$ 。

评注. 每次询问区间内所有子区间的 mex 的和也是能做的。以及 CF1148H 查询有多少个子区间满足 mex $= k$ ， k 每次不同。也可以使用极小 mex 区间机械地做。

例 5.33. CF997E Good Segments. 给一个排列，每次询问一个区间有多少子区间满足 $\max - \min = r - l$ 。提示

例 5.34. 带有询问矩形的矩形面积并。 矩形并局限在一个询问矩形内的面积。

例 5.35. NOIP 2022 比赛. 给定两个序列 a, b ，每次询问

$$\sum_{[i,j] \subseteq [l,r]} \max a[i,j] \cdot \max b[i,j].$$

5.5 维护时间维

例 5.36. 维护序列，每个位置是一个栈，初始均为空。 支持对区间内的栈压入一个分治信息；查询只考虑 $[l, r]$ 时刻的修改时，第 i 个栈中的信息和。

评注. 这个问题的强制在线需要一些聪明的手段。

例 5.37. IOI2021 分糖果. 维护序列 a_i ，每个位置有一个上限 c_i 。支持：区间加（可能有负数）并在加完之后让 a_i 裁剪到 $[0, c_i]$ （即小于 0 变成 0，大于 c_i 变成 c_i ），查询单点值。

练习 5.38. P7560

例 5.39. 给一个序列 a ，支持区间加，查询只考虑第 l 到第 r 个修改时的区间和。

例 5.40. P8512. 给一个初值为 0 的序列 a ，支持区间赋值，查询只考虑第 l 到第 r 个修改时的区间和。

6 分治

6.1 偏序数点

在前一章中我们提到过，可以用分治来查询偏序范围最值乃至任意分治信息。我们在此给出做法。

不妨假设范围是 d 个正交半平面围出来的，也就是每个点是一个 d 维带权点 $\mathbf{x}_i$ ，每次查询 $\mathbf{x}_i \leq \mathbf{y}$ 的所有点的分治信息和。

由于这里没有顺序，需要分治信息可交换

我们对第一维进行分治： $\text{solve}(l, r)$ 处理所有 $x_1, y_1 \in [l, r]$ 的那些点与查询之间的贡献。那么 $\text{solve}(l, r)$ 可以拆成三部分： $\text{solve}(l, m)$ ， $\text{solve}(m+1, r)$ ，以及处理所有 $x_1 \in [l, m]$ ， $y_1 \in [m+1, r]$ 的那些点与查询之间的贡献。注意到第三部分变成了一个相同规模但是维数减一的问题，可以递归到维数减一的问题。来算一下复杂度：设 $T(d, n, V)$ 是维数为 d ，规模为 n ，值域大小为 V 时 solve 的复杂度，那么 $T(d, n, V) \leq T(d, n_l, V/2) + T(d, n_r, V/2) + T(d-1, n, V_0) + O(n)$ 。

这里的初值是关键。考虑 $d = 1$ 的情况，如果直接按照分治的方法，那么我们需要 $O(n \log V)$ 的时间；而如果事先排好序了，那么就只需要 $O(n)$ 的时间了。所以，取决于点是否已经排好序了， $T(1, n, V)$ 为 $O(n)$ 或者 $O(n \log n)$ 。 $d = 2$ 的时候则是标准的二维数点，可以通过上一章中的扫描线方法做到 $O(n \log n)$ 。不过，我们在这里也介绍一种纯分治的方法。由于当点排好序时， $T(1, n, V) = O(n \log n)$ ，我们可以让 $\text{solve}(l, r)$ 顺便支持将其内的所有点和询问按照第二维排序。这样，在递归处理完两个子问题之后，就可以 $O(n)$ 计算跨过中点的贡献，并且可以通过归并来 $O(n)$ 排序整个区间。这样我们也得到 $T(2, n, V) = T(2, n_l, V/2) + T(2, n_r, V/2) + O(n) = O(n \log V)$ 。对于更大的 d ，使用前面的递归式即可算出 $T(d, n, V) = O(n \log^{d-1} V)$ 。另外，由于 V 总是可以通过离散化变成 $O(n)$ ，我们可以在 $O(n \log^{d-1} n)$ 的时间内解决这个问题。

评注. 当分治结构与询问无关时，将分治过程记录下来即可得到支持在线询问的数据结构，即树套树。所以，这个问题也存在 $O(\log^{d-1} n)$ 的在线做法，只不过这超出了大纲的范围。

在实践中， d 几乎总是不超过 3，更大的情况通常需要寻找其他方法。通常来说，分治方法在 $d = 2$ 时的子问题的处理是最关键的。

比如数点的分块 bitset $O(n^2/w)$ ，一般情况的 KD tree $O(n^{2-1/d})$

练习 6.1. 二维偏序. 计算一个序列的逆序对数量。

练习 6.2. 三维偏序. 给定序列 a, b, c ，查询满足 $a_i \leq a_j, b_i \leq b_j, c_i \leq c_j$ 的 (i, j) 的数量。

练习 6.3. 带修二维数点. 二维平面，支持插入或删除一个带权点 $\mathbf{x}$ ，查询所有 $\mathbf{x} \leq \mathbf{y}$ 的点的分治信息和。

练习 6.4. P3157. 每次删除一个数，查询剩下的序列的逆序对数

练习 6.5. P4169. 二维平面，支持插入一个点，查询离一个点最近的点到它的距离，距离为曼哈顿距离 $|\Delta x| + |\Delta y|$ 。

6.1.1 半在线

我们的分治过程事实上可以支持一种比较弱的在线操作：假设点和询问初始全部已知，且在第一维上有序，但点权不是在最开始即给定的，而是在回答询问后才给出。比如说，给定 $(1, x_1)$ 的点权，在回答询问 $(2, y_2)$ 后再给定 $(3, x_3)$ 的点权，然后再在回答询问 $(4, y_4)$ 后给定 $(5, x_5)$ 的点权，以此类推。由于计算跨过中点的贡献时只需要左边的点权全部已知，我们可以先递归左边，然后处理跨过中点的贡献，再递归右边。这在 OI 里通常被称为 **cdq 分治**。

练习 6.6. P2487. 二维偏序意义下的 LDS，还需要计数并计数每个点在多少种方案中出现。

练习 6.7. P4093.

OI 中最早是陈丹琦推广的。但是没有人正式定义过 cdq 分治到底指的是什么。